Tries & Suffix Tries

Ben Langmead

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

Department of Computer Science

CC BY

Please sign guestbook (www.langmead-lab.org/teaching-materials) to tell me briefly how you are using the slides. For original Keynote fles, email me (ben.langmead@gmail.com).

Tries

A trie ("try") is a tree representing a collection of strings (keys): the smallest tree such that

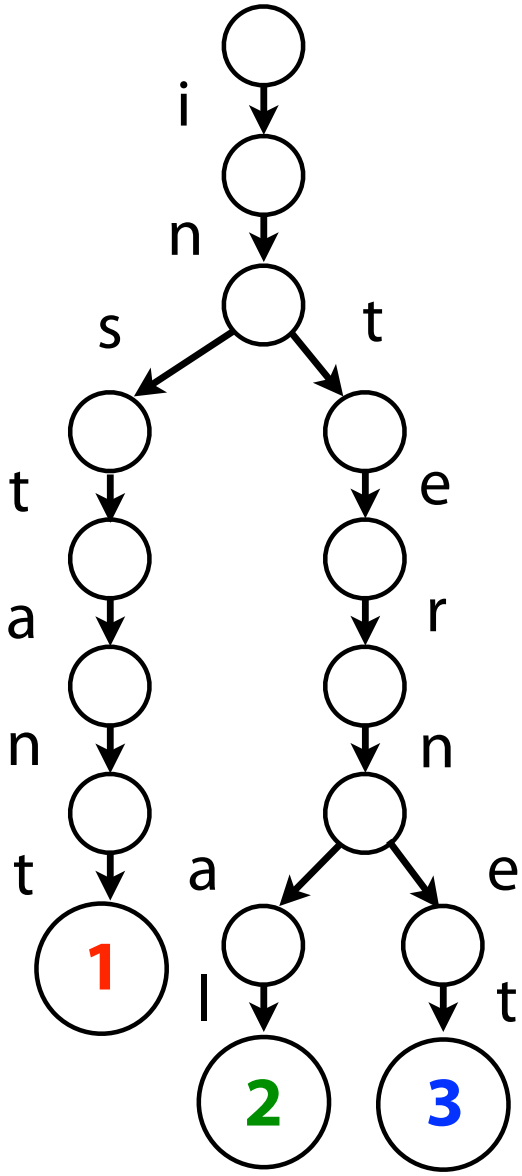
- Each edge is labeled with a character $c \in \Sigma$
- For given node, at most one child edge has label c, for any $c \in \Sigma$
- Each key is "spelled out" along some path starting at root

Helpful for implementing a *set* or *map* when the keys are strings

Tries

Keys: instant, internal, internet

Key	Value
instant	1
internet	2
internet	3



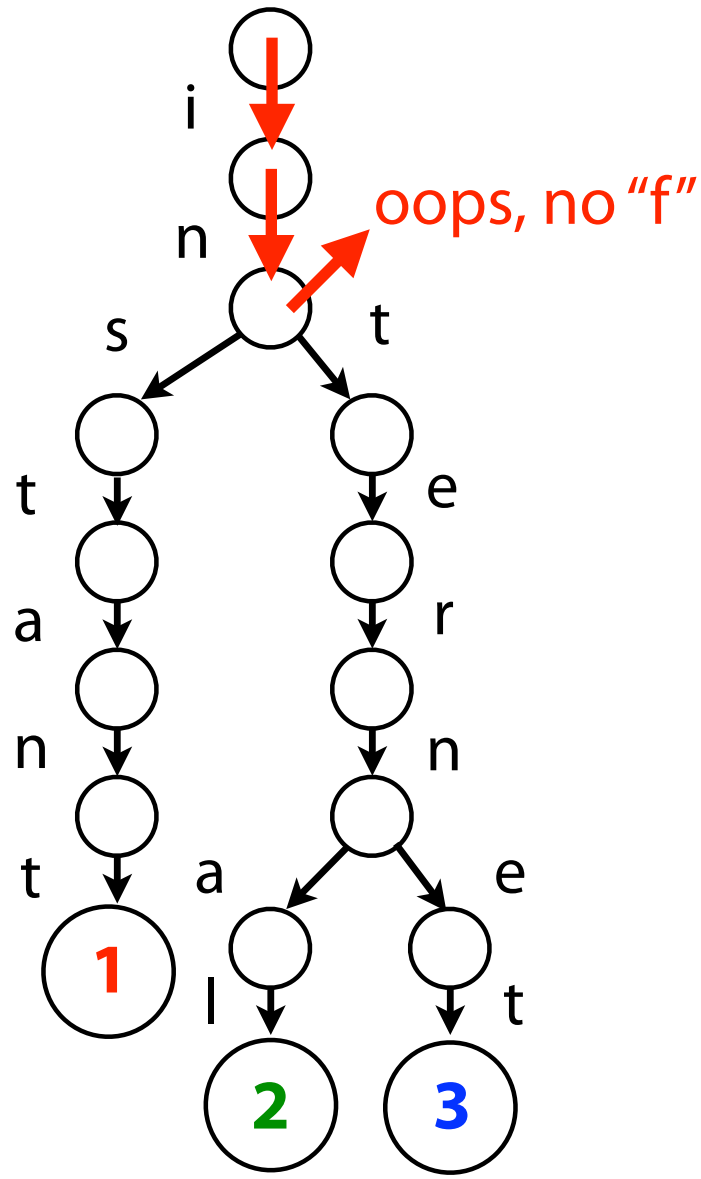
Smallest tree such that:

- Each edge is labeled with a character $c \in \Sigma$
- For given node, at most one child edge has label c, for any $c \in \Sigma$
- Each key is "spelled out" along some path starting at root

Tries

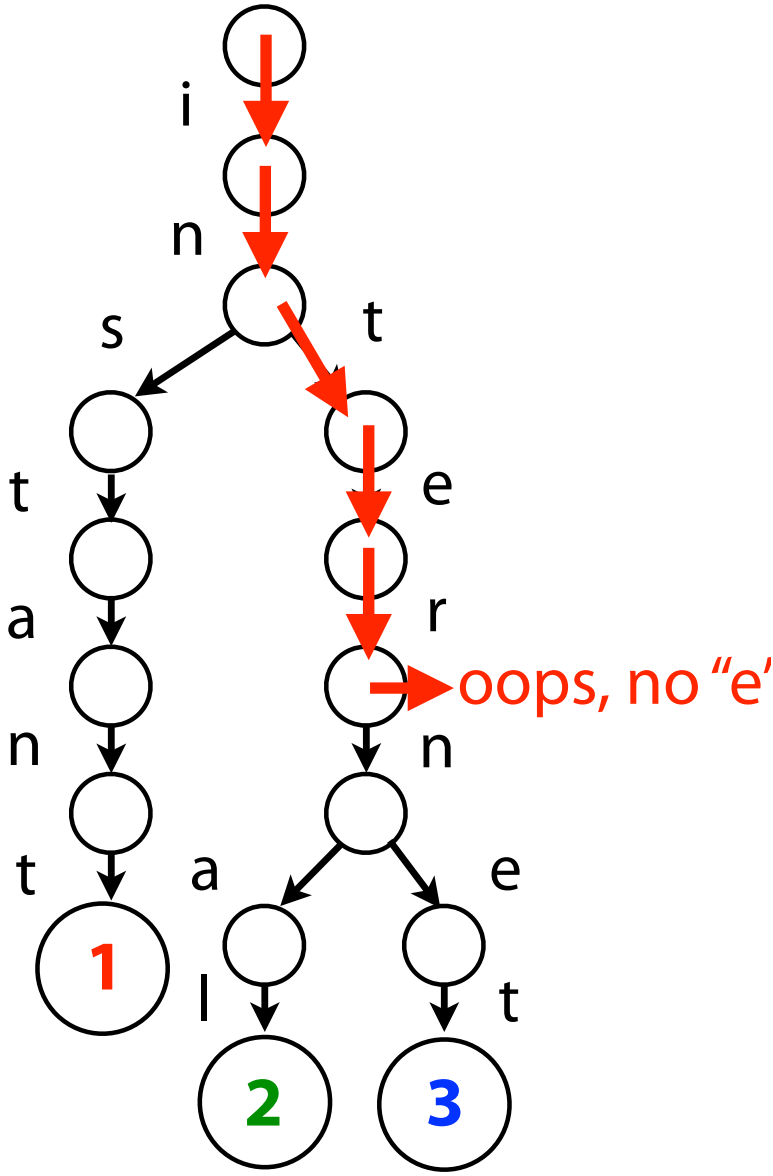
How do we check whether "infer" is in the trie?

Start at root and try to match successive characters of "infer" to edges in trie



Tries

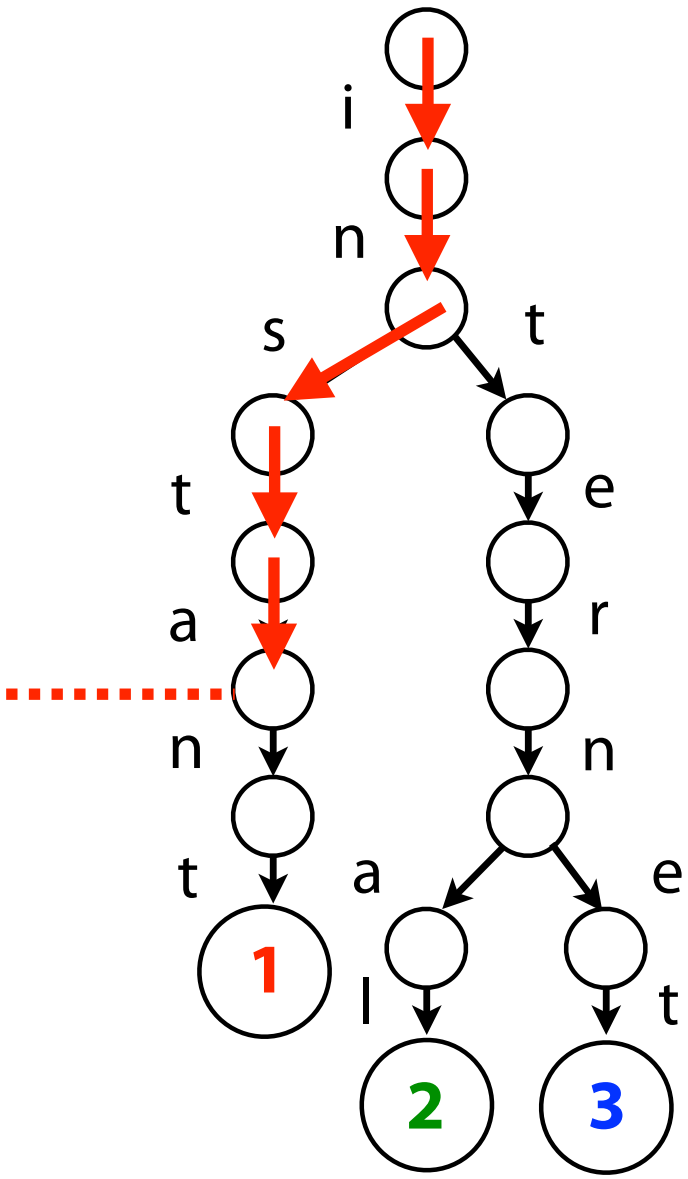
Matching “interesting”



Tries

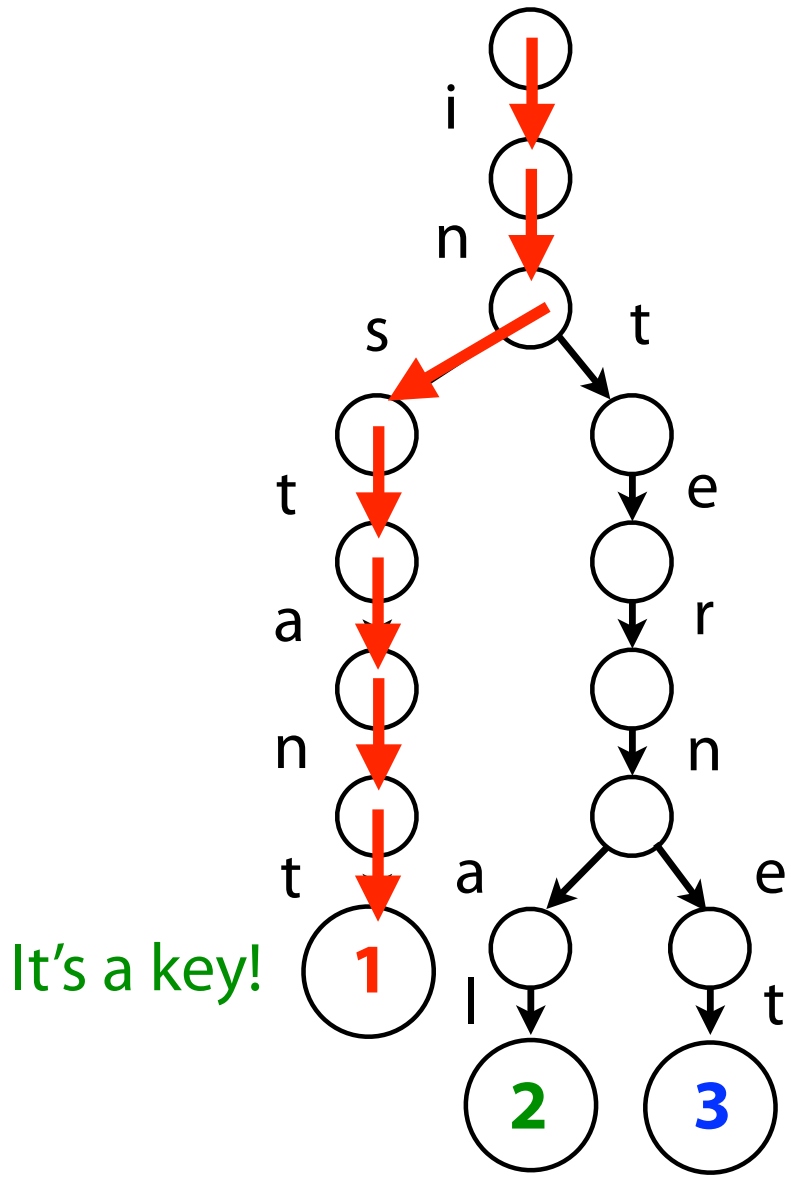
Matching “insta”

No value associated with node, so “insta” wasn’t a key

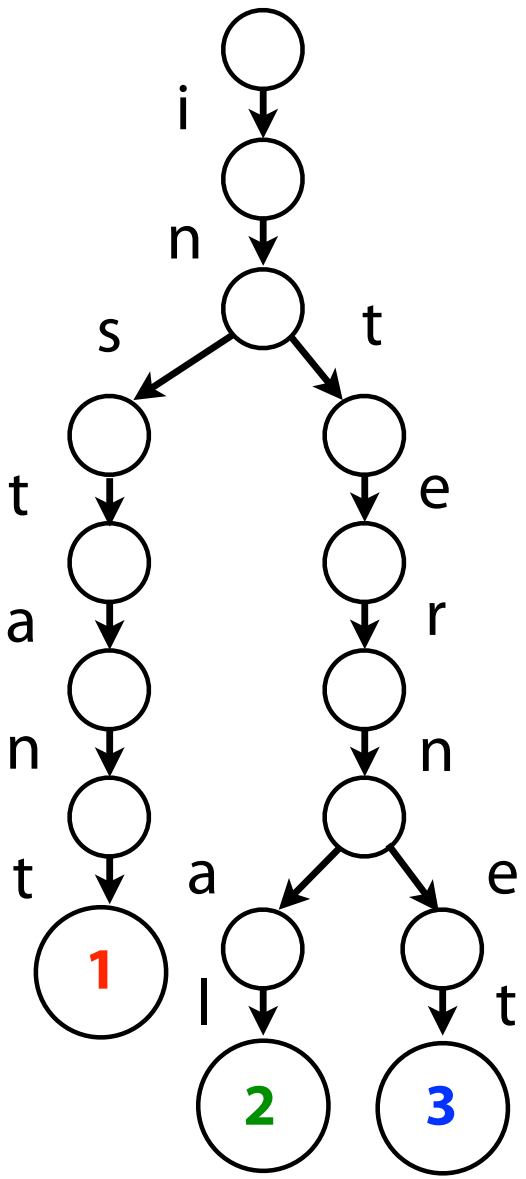


Tries

Matching “instant”



Tries



Checking for presence of key P, where $|P| = n$ traverses $\leq$ $\boldsymbol{n}$ edges

If total length of all keys is N, trie has $\leq$ $\boldsymbol{N}$ edges

Suffix trie

Build a **trie** containing all **suffixes** of a text *T*

```
T: GTTATAGCTGATCGCGGCGTAGCGG$
   GTTATAGCTGATCGCGGCGTAGCGG$
    TTATAGCTGATCGCGGCGTAGCGG$
     TATAGCTGATCGCGGCGTAGCGG$
      ATAGCTGATCGCGGCGTAGCGG$
       TAGCTGATCGCGGCGTAGCGG$
        AGCTGATCGCGGCGTAGCGG$
         GCTGATCGCGGCGTAGCGG$
          CTGATCGCGGCGTAGCGG$
           TGATCGCGGCGTAGCGG$
            GATCGCGGCGTAGCGG$
             ATCGCGGCGTAGCGG$
              TCGCGGCGTAGCGG$
               CGCGGCGTAGCGG$
                GCGGCGTAGCGG$
                 CGGCGTAGCGG$
                  GGCGTAGCGG$
                   GCGTAGCGG$
                    CGTAGCGG$
                     GTAGCGG$
                      TAGCGG$
                       AGCGG$
                        GCGG$
                         CGG$
                          GG$
                           G$
```

m(m+1)/2 chars

Suffix trie

First add special *terminal character* **$** to the end of *T*

$ is a character that does not appear elsewhere in *T*, and we define it to be less than other characters (**$** < **A** < **C** < **G** < **T**)

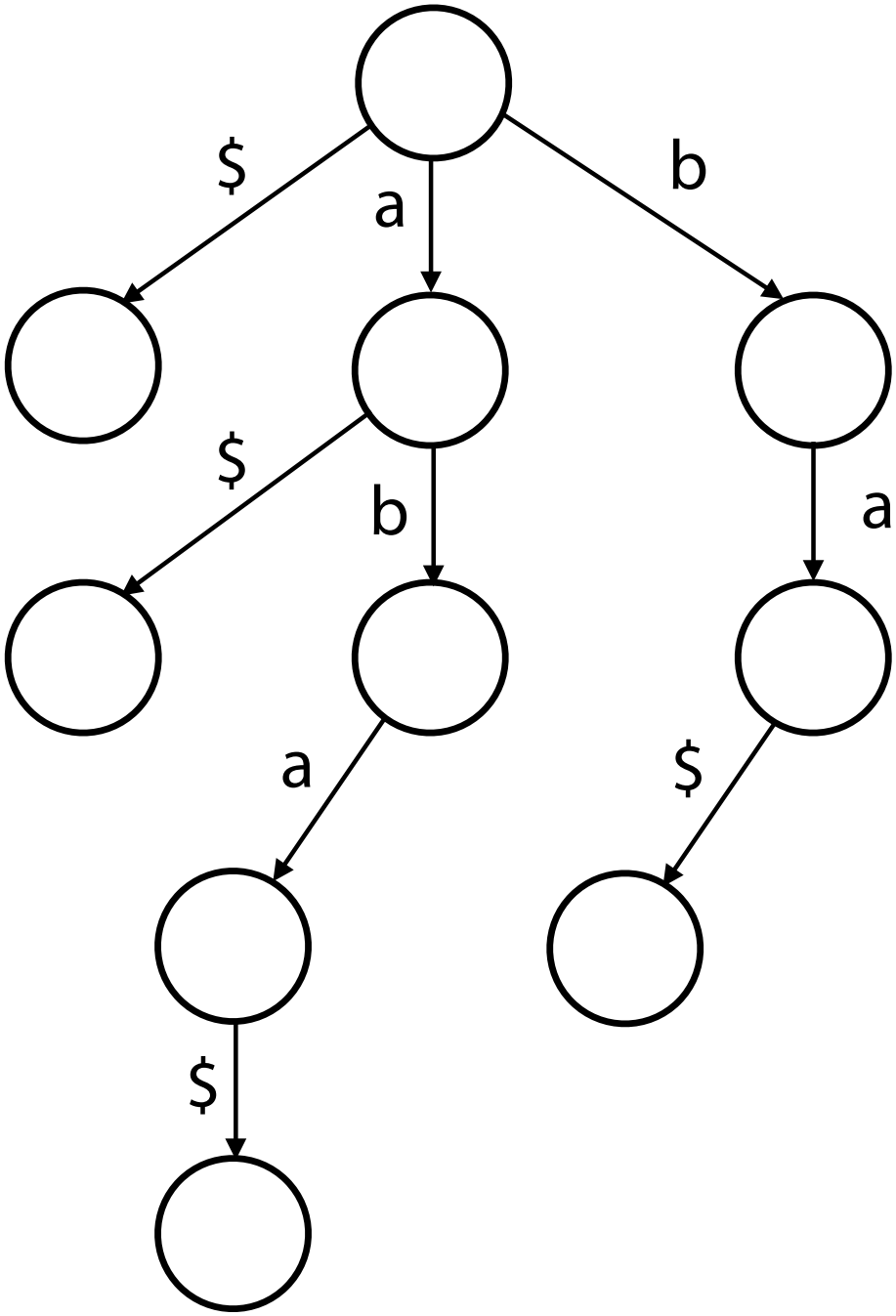
$ enforces a familiar rule: e.g. "as" comes before "ash" in the dictionary.
$ also guarantees no suffix is a prefix of any other suffix.

```
T: GTTATAGCTGATCGCGGCGTAGCGG$
   GTTATAGCTGATCGCGGCGTAGCGG$
    TTATAGCTGATCGCGGCGTAGCGG$
     TATAGCTGATCGCGGCGTAGCGG$
      ATAGCTGATCGCGGCGTAGCGG$
       TAGCTGATCGCGGCGTAGCGG$
        AGCTGATCGCGGCGTAGCGG$
         GCTGATCGCGGCGTAGCGG$
          CTGATCGCGGCGTAGCGG$
           TGATCGCGGCGTAGCGG$
            GATCGCGGCGTAGCGG$
             ATCGCGGCGTAGCGG$
              TCGCGGCGTAGCGG$
               CGCGGCGTAGCGG$
```

Suffix trie

T: aba**$**

What's the suffix trie?



Suffix trie

Excerpt from SuffixTrie class: http://bit.ly/CG_SuffixTrie

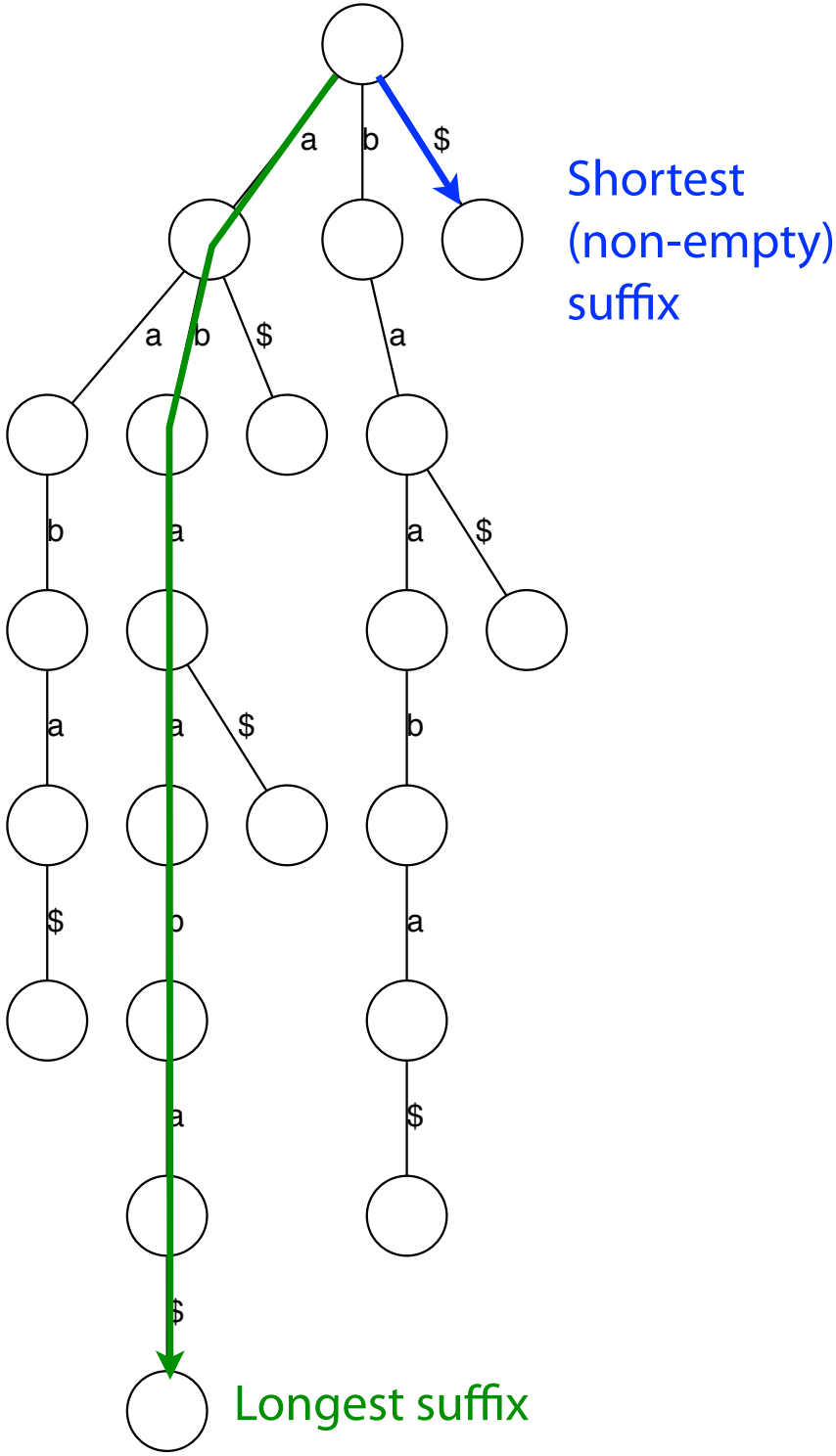
```
def __init__(self, t):
    """ Make suffix trie from t """
    t += '$'                    # add terminator
    self.root = {}
    for i in range(len(t)):     # for each suffix
        cur = self.root
        for c in t[i:]:         # for each character in i'th suffix
            if c not in cur:
                cur[c] = {}     # add outgoing edge if necessary
            cur = cur[c]        # follow the edge and continue
```

Suffix trie

T: abaaba**$**

Each path from root to leaf represents a suffix; each suffix is represented by some path from root to leaf

Would this still be the case if we hadn't added **$**?

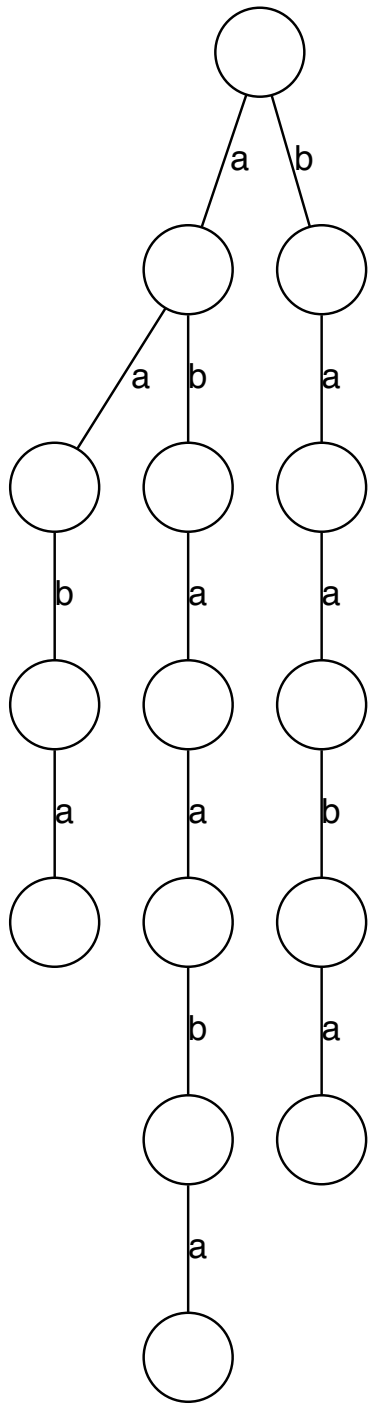


Suffix trie

T: abaaba**$**

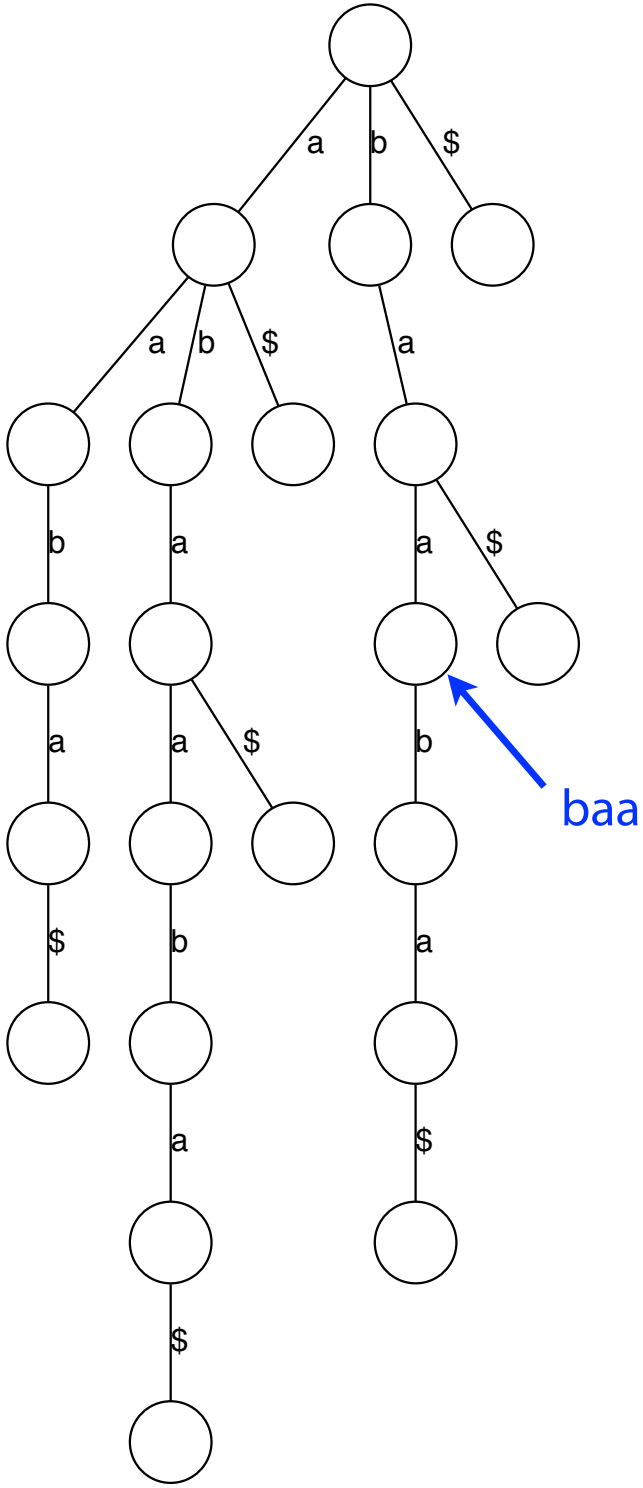
Each path from root to leaf represents a suffix; each suffix is represented by some path from root to leaf

Would this still be the case if we hadn't added **$**? **No**



Suffix trie

Think of each node as having a **label**, spelling out characters on path from root to node



Suffix trie

How do we check whether a string *S* is a substring of *T*?

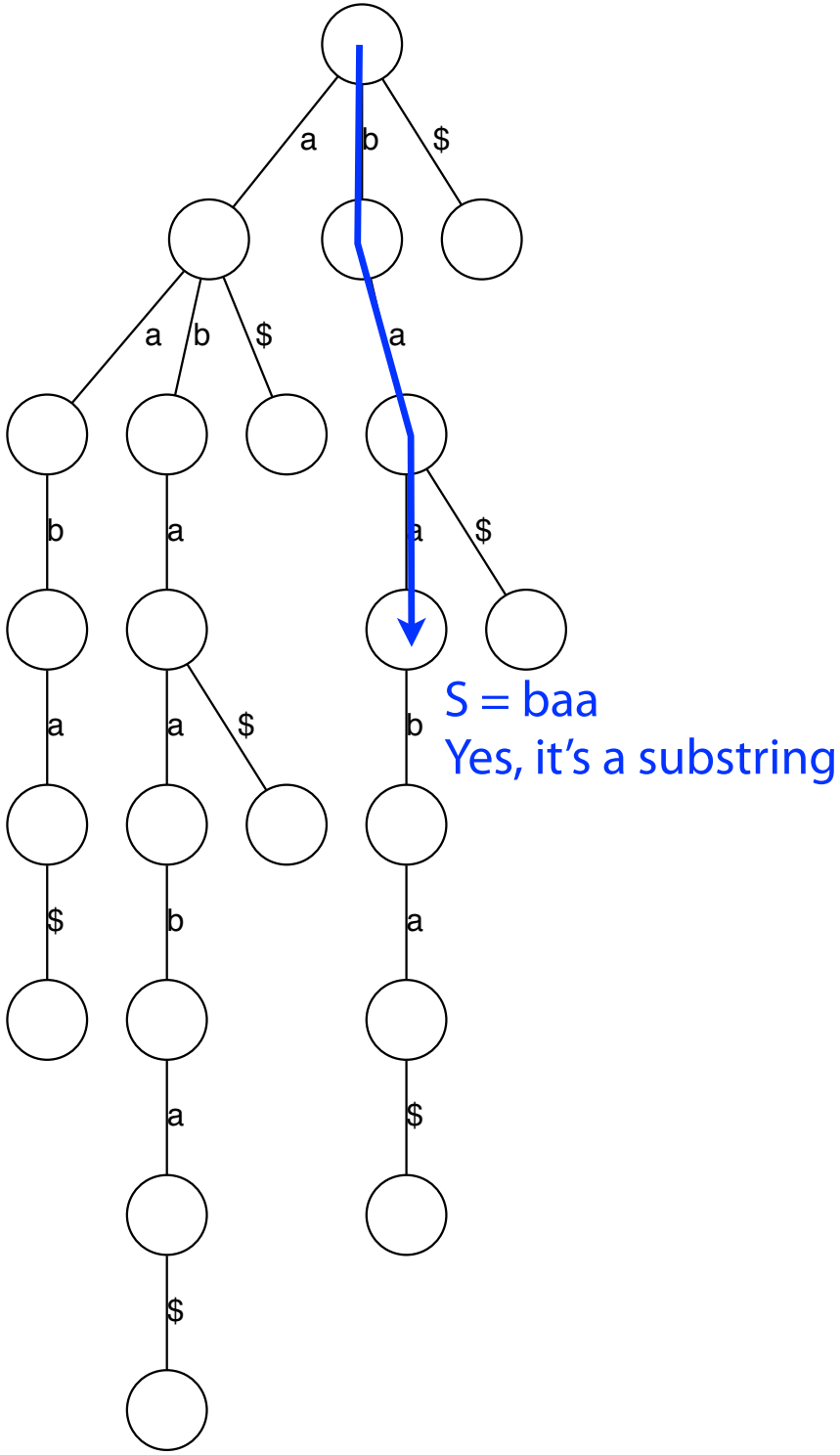
Note: Each of *T*'s substrings is spelled out along a path from the root.

Every *substring* is a *prefix* of some *suffix* of T.

Start at the root and follow the edges labeled with the characters of *S*

If we "fall off" the trie -- i.e. there is no outgoing edge for next character of *S*, then *S* is not a substring of *T*

If we exhaust *S* without falling off, *S* is a substring of *T*



Suffix trie

How do we check whether a string *S* is a substring of *T*?

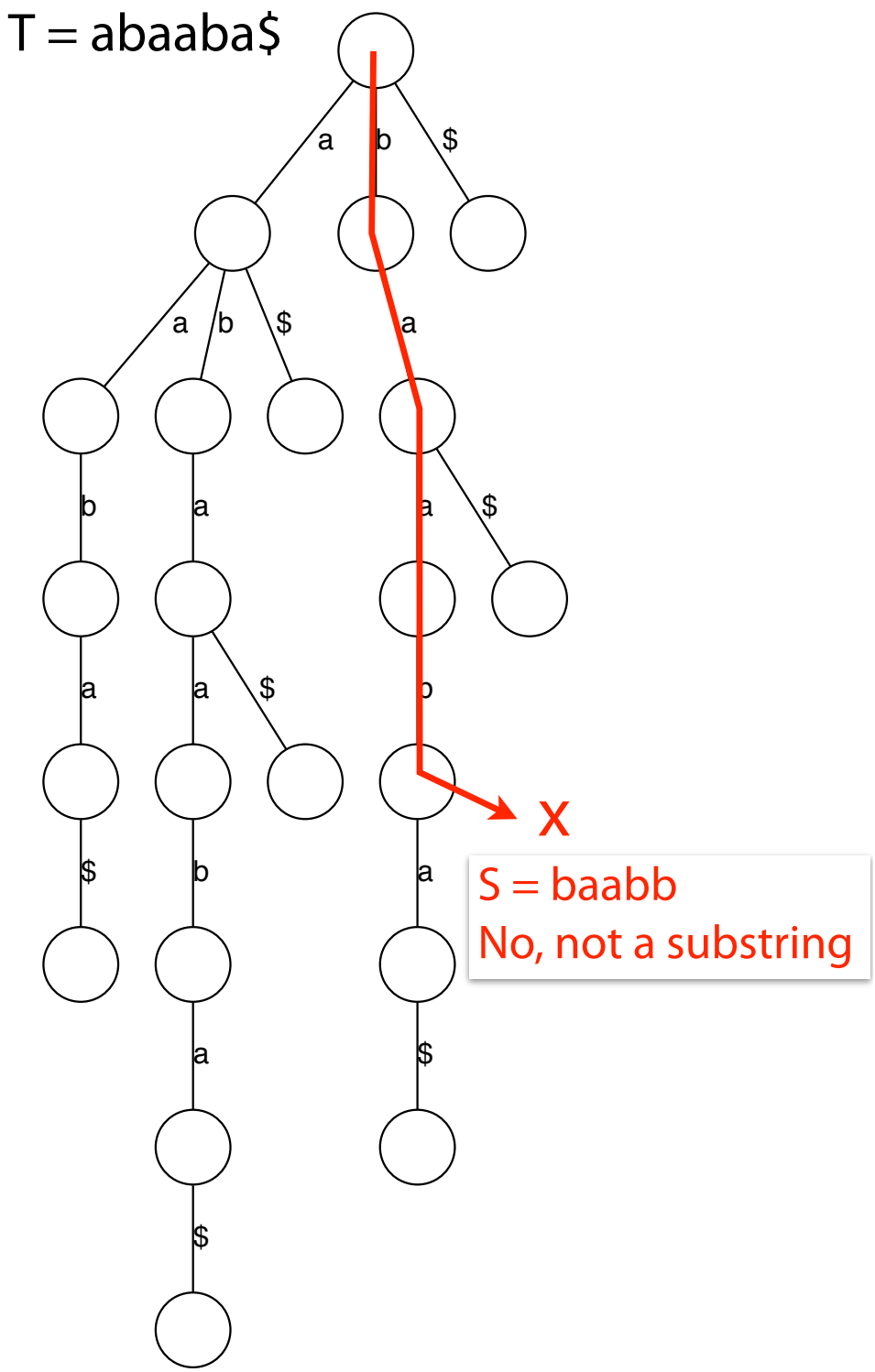
Note: Each of *T*'s substrings is spelled out along a path from the root.

Every *substring* is a *prefix* of some *suffix* of T.

Start at the root and follow the edges labeled with the characters of *S*

If we "fall off" the trie -- i.e. there is no outgoing edge for next character of *S*, then *S* is not a substring of *T*

If we exhaust *S* without falling off, *S* is a substring of *T*



Suffix trie

Excerpt from SuffixTrie class: http://bit.ly/CG_SuffixTrie

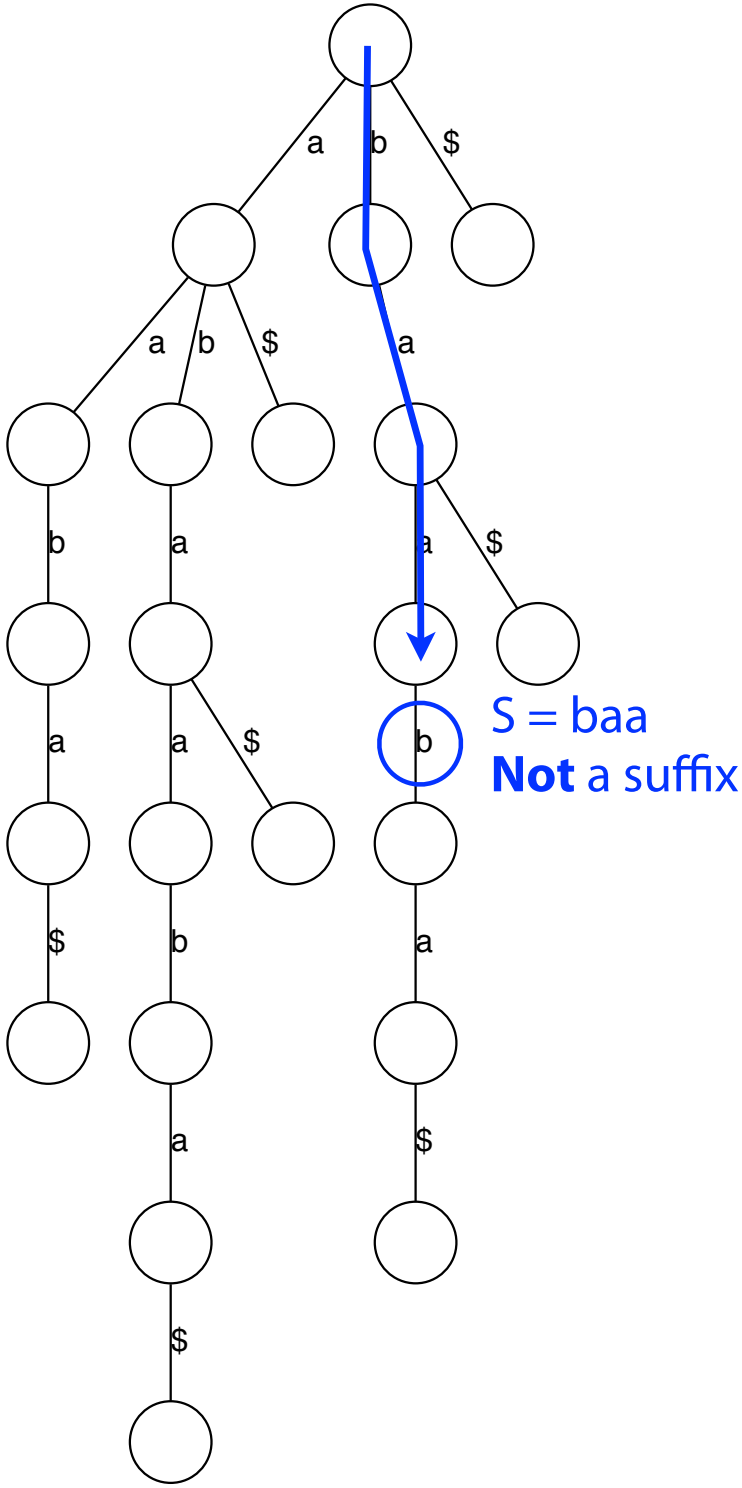
```
def follow_path(self, s):
    """ Follow path given by characters of s.  Return node at
        end of path, or None if we fall off. """
    cur = self.root
    for c in s:
        if c not in cur:
            return None  # no outgoing edge on next character
        cur = cur[c]     # descend one level
    return cur

def has_substring(self, s):
    """ Return true if s appears as a substring of t """
    return self.follow_path(s) is not None
```

Suffix trie

How do we check whether a string *S* is a **suffix** of *T*?

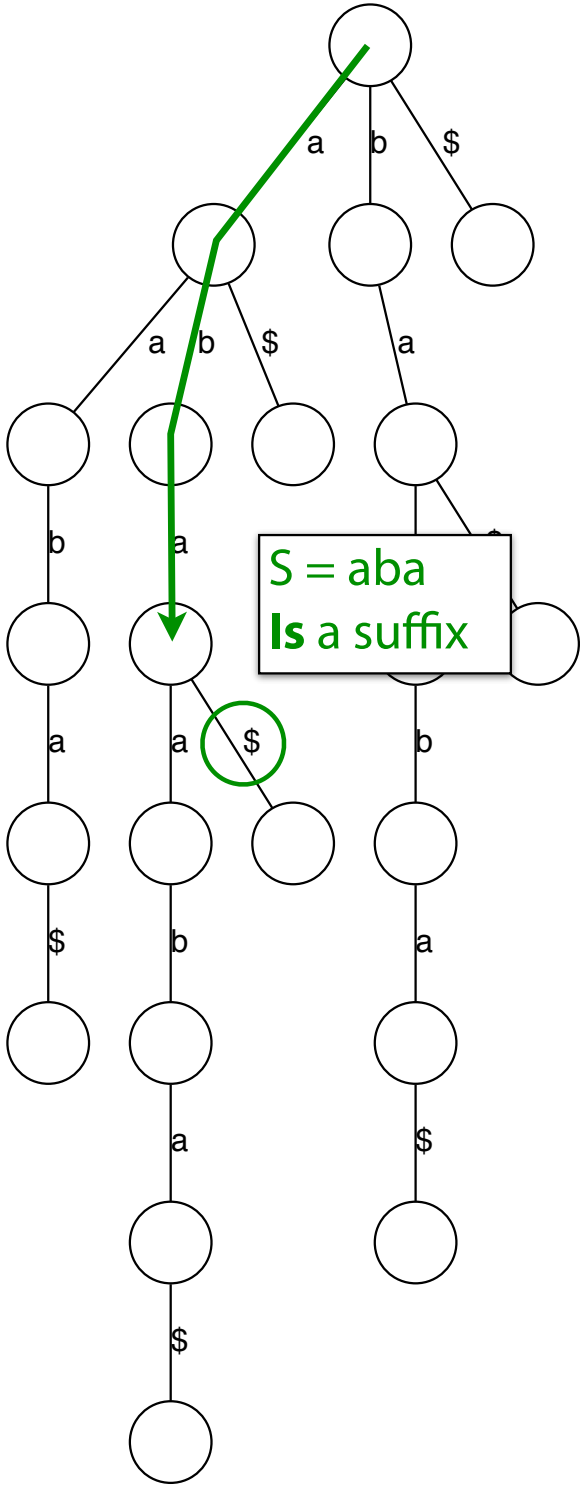
Same procedure as for substring, but additionally check terminal node for **$** child



Suffix trie

How do we check whether a string *S* is a **suffix** of *T*?

Same procedure as for substring, but additionally check terminal node for **$** child

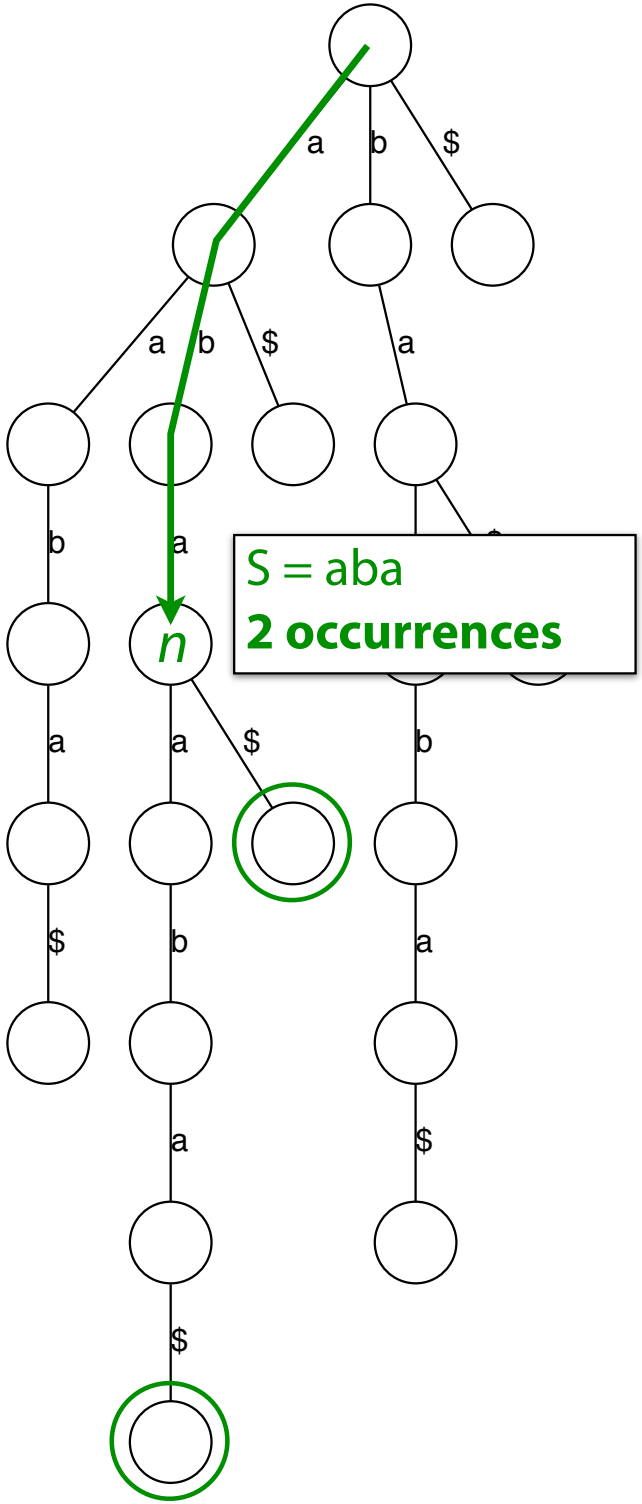


Suffix trie

How do we count the **number of times** a string *S* occurs as a substring of *T*?

Follow path labeled with *S*. If we fall off, answer is 0. If we end up at node *n*, answer equals # of leaves in subtree rooted at *n*.

Leaves can be counted with depth-first traversal.



Suffix trie

How does the suffix trie grow with $|T| = m$?

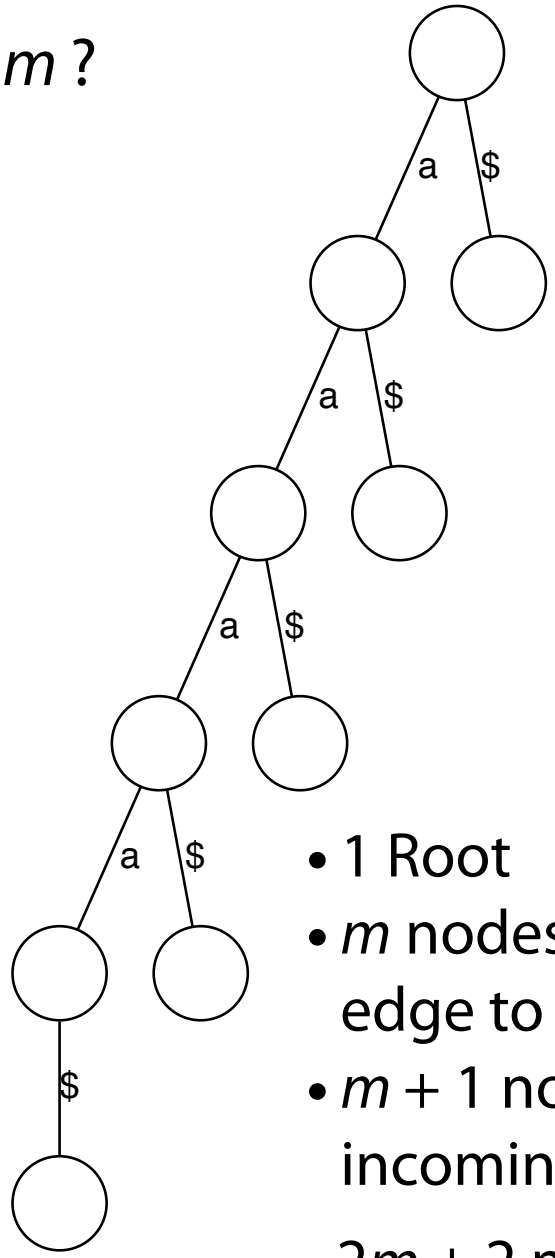
Suffix trie

How does the suffix trie grow with $|T| = m$?

Is there a class of string where the number of suffix trie nodes grows linearly with m?

Yes: a string of m a's in a row (a^m)

T = aaaa$



- 1 Root
- m nodes with "a" edge to parent
- $m + 1$ nodes with incoming **$** edge

$2m + 2$ nodes

Suffix trie

T = aaabbb$

How does the suffix trie grow with $|T| = m$?

Is there a class of string where the number of suffix trie nodes grows with m^2 ?

Yes: $a^n b^n$ where $2n = m$

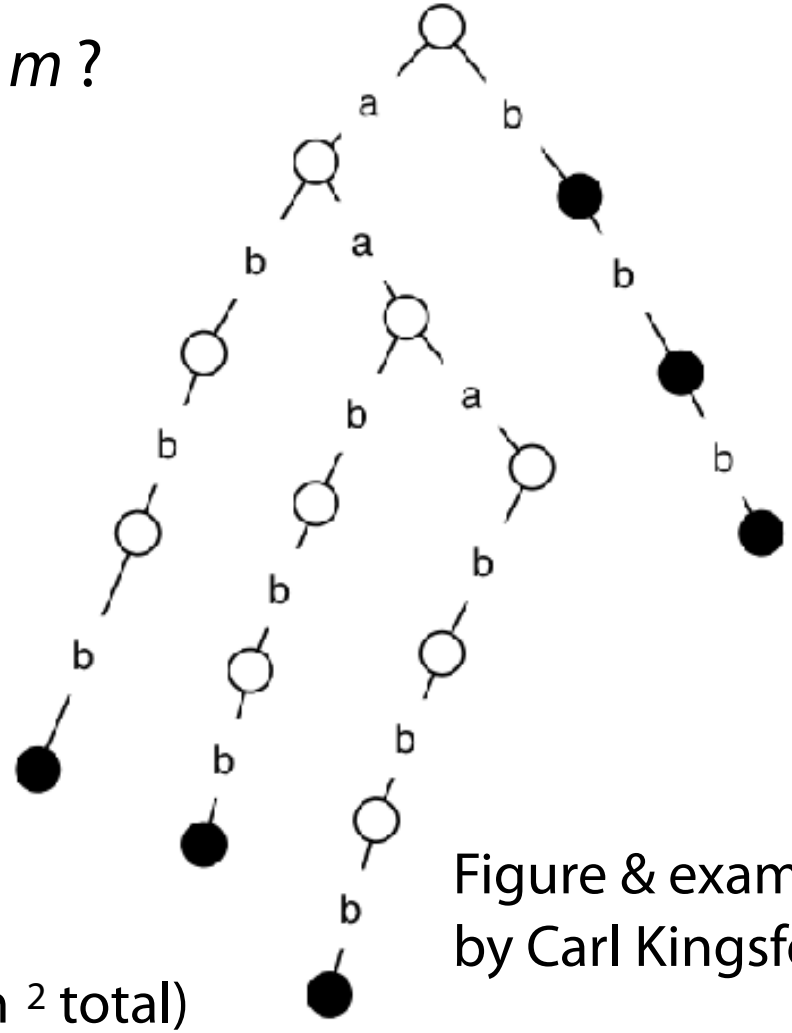


Figure & example by Carl Kingsford

- 1 root
- n nodes along "b chain," right
- n nodes along "a chain," middle
- n chains of n "b" nodes hanging off "a chain" (n^2 total)
- $2n + 1$ **$** leaves (not shown)

$n^2 + 4n + 2$ nodes, where $m = 2n$

Suffix trie: upper bound on size: idea 1

Recall our function for build building a suffix trie

```
def __init__(self, t):
    """ Make suffix trie from t """
    t += '$'
    self.root = {}
    for i in range(len(t)):
        cur = self.root
        for c in t[i:]:
            if c not in cur:
                cur[c] = {}
            cur = cur[c]
```

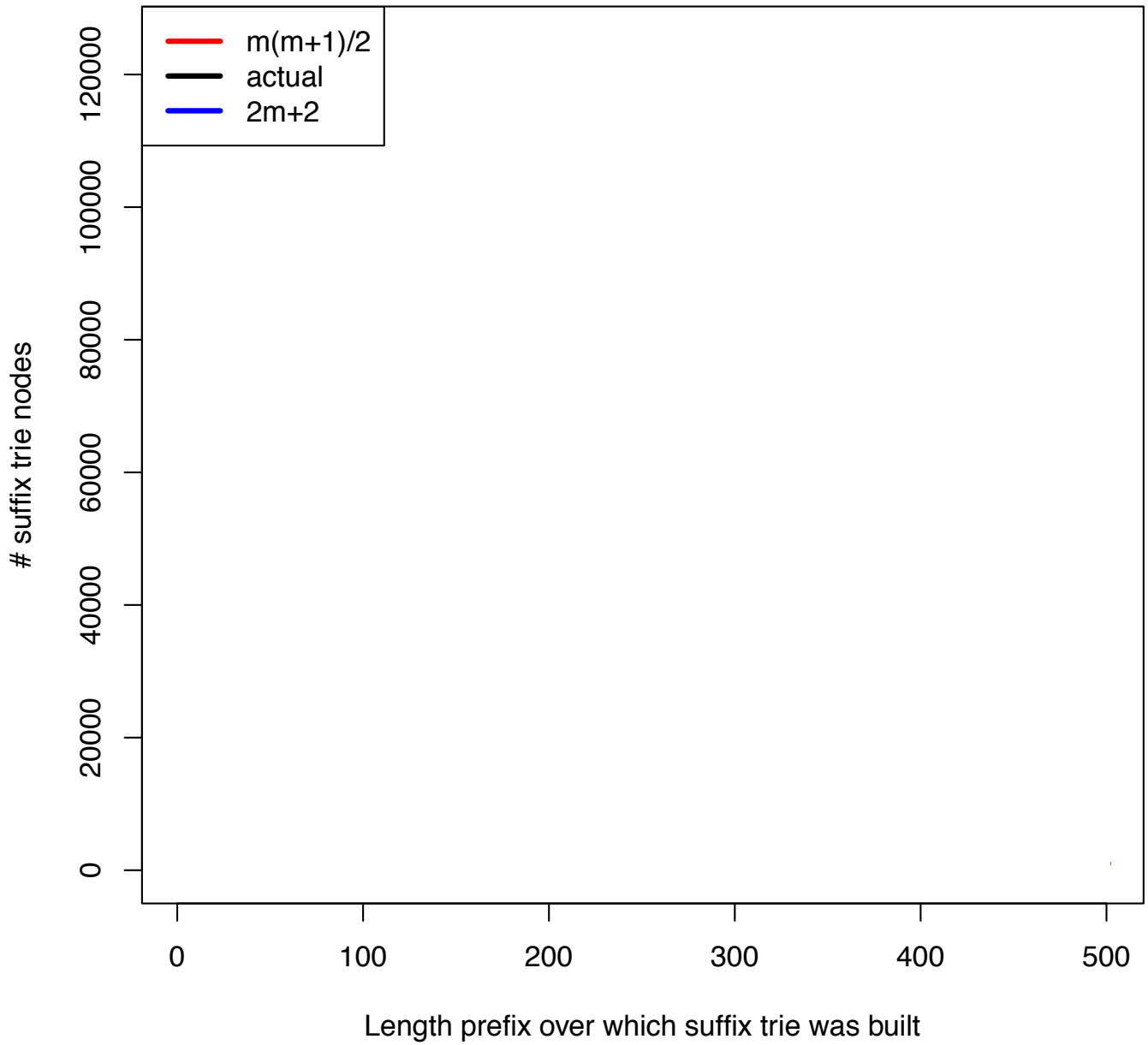
Adds 1 node, runs at most $m(m+1)/2$ times

```
GTTATAGCTGATCGCGGCGTAGCGG$
GTTATAGCTGATCGCGGCGTAGCGG$
 TTATAGCTGATCGCGGCGTAGCGG$
  TATAGCTGATCGCGGCGTAGCGG$
   ATAGCTGATCGCGGCGTAGCGG$
    TAGCTGATCGCGGCGTAGCGG$
     AGCTGATCGCGGCGTAGCGG$
      GCTGATCGCGGCGTAGCGG$
       CTGATCGCGGCGTAGCGG$
        TGATCGCGGCGTAGCGG$
         GATCGCGGCGTAGCGG$
          ATCGCGGCGTAGCGG$
           TCGCGGCGTAGCGG$
            CGCGGCGTAGCGG$
             GCGGCGTAGCGG$
              CGGCGTAGCGG$
               GGCGTAGCGG$
                GCGTAGCGG$
                 CGTAGCGG$
                  GTAGCGG$
                   TAGCGG$
                    AGCGG$
                     GCGG$
                      CGG$
                       GG$
                        G$
```

Suffix trie: actual growth

Built suffix tries for the first 500 prefixes of the lambda phage virus genome

Black curve shows how # nodes increases with prefix length

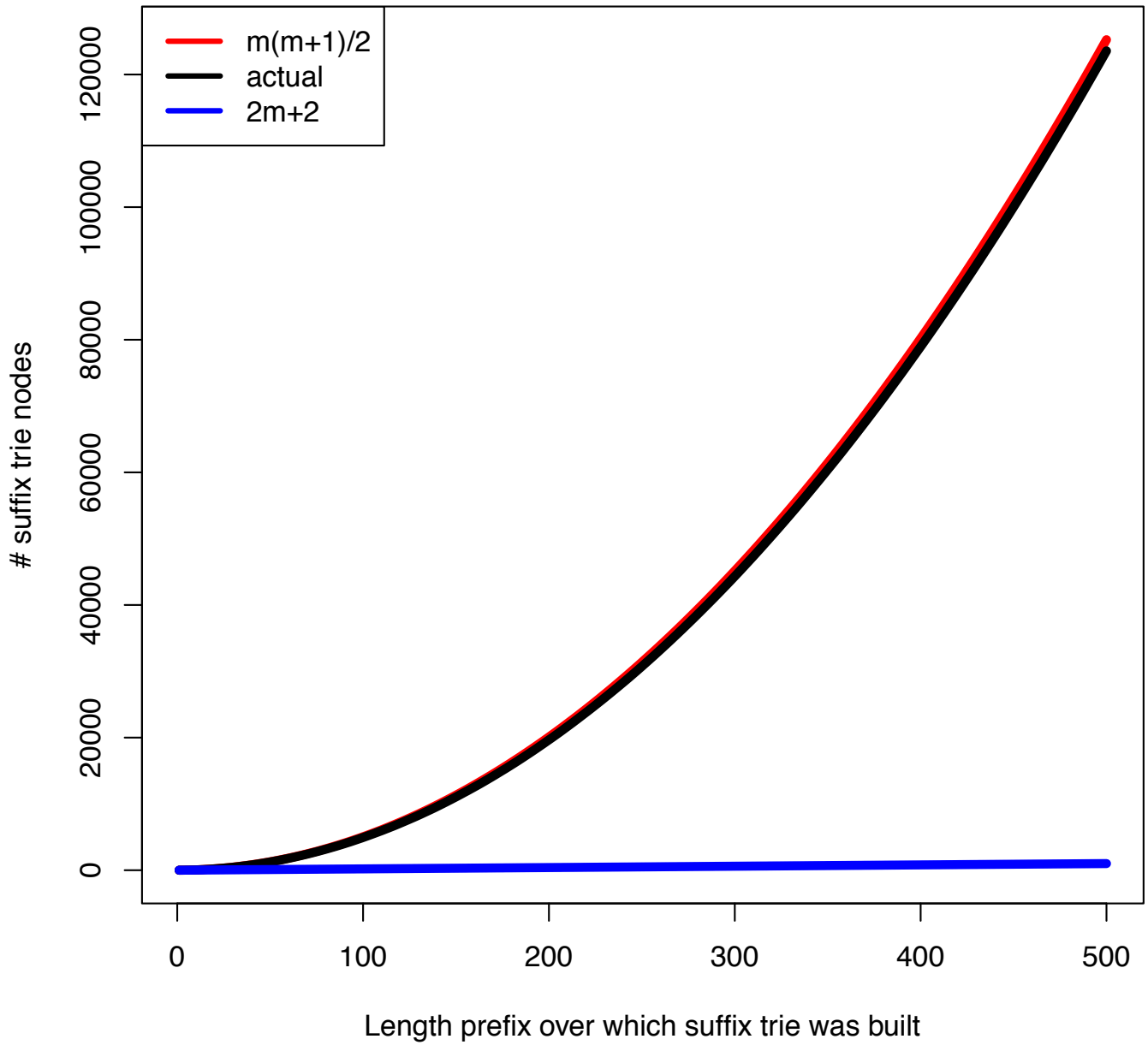


Suffix trie: actual growth

Built suffix tries for the first 500 prefixes of the lambda phage virus genome

Black curve shows how # nodes increases with prefix length

Actual growth *much* closer to worst case than to best!



Suffix Trees

Ben Langmead

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

Department of Computer Science

CC BY

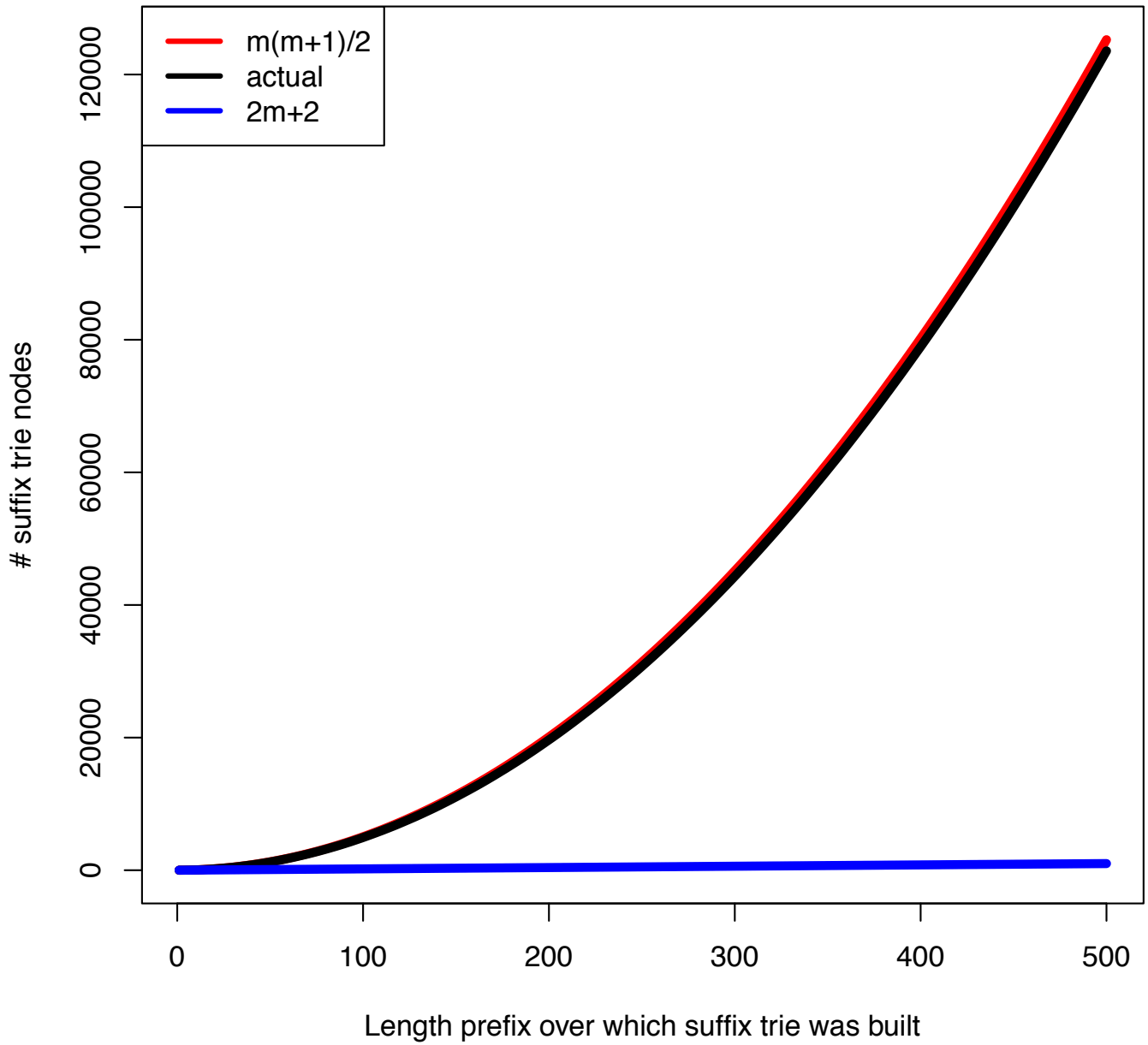
Please sign guestbook (www.langmead-lab.org/teaching-materials) to tell me briefly how you are using the slides. For original Keynote fles, email me (ben.langmead@gmail.com).

Suffix trie

We saw the suffix trie, but we also saw its size grows *quadratically* with the length of the string

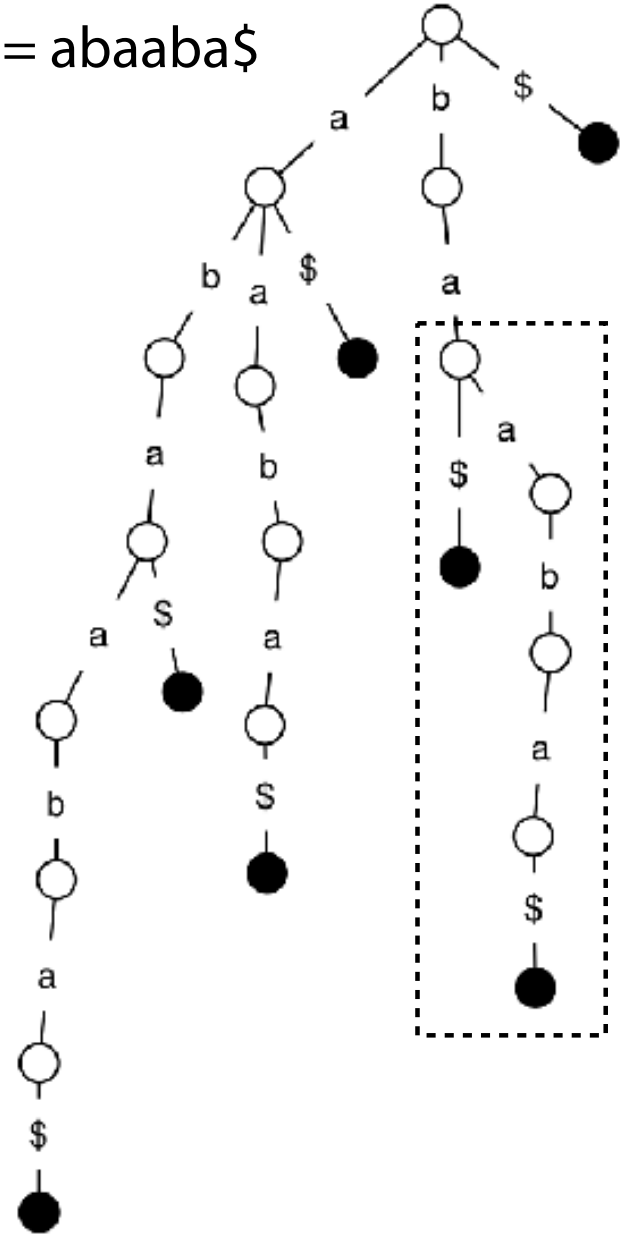
Human genome is $3 \cdot 10^9$ bases long.

If $m = 3 \cdot 10^9$, m^2 is way huge, far beyond what we can store in memory



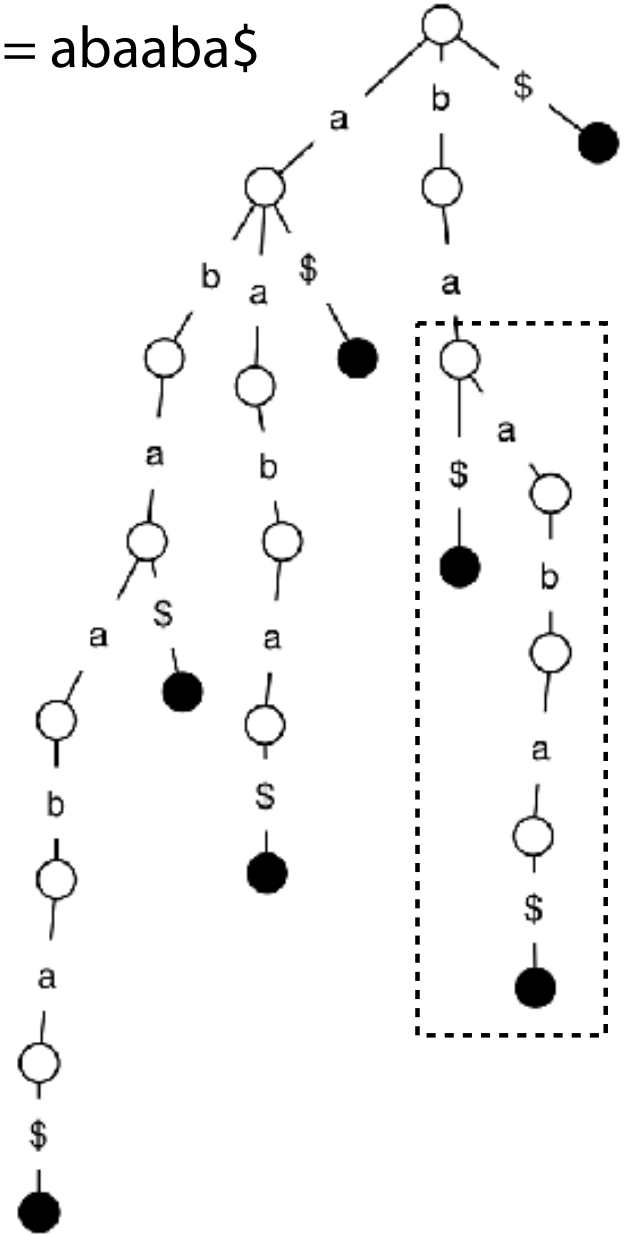
Suffix trie: making it smaller

T = abaaba$

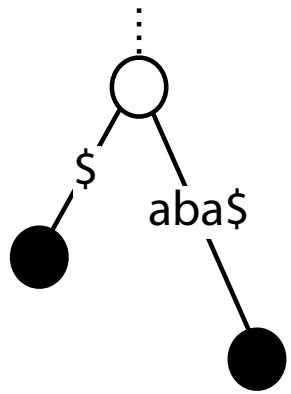


Suffix trie: making it smaller

T = abaaba$



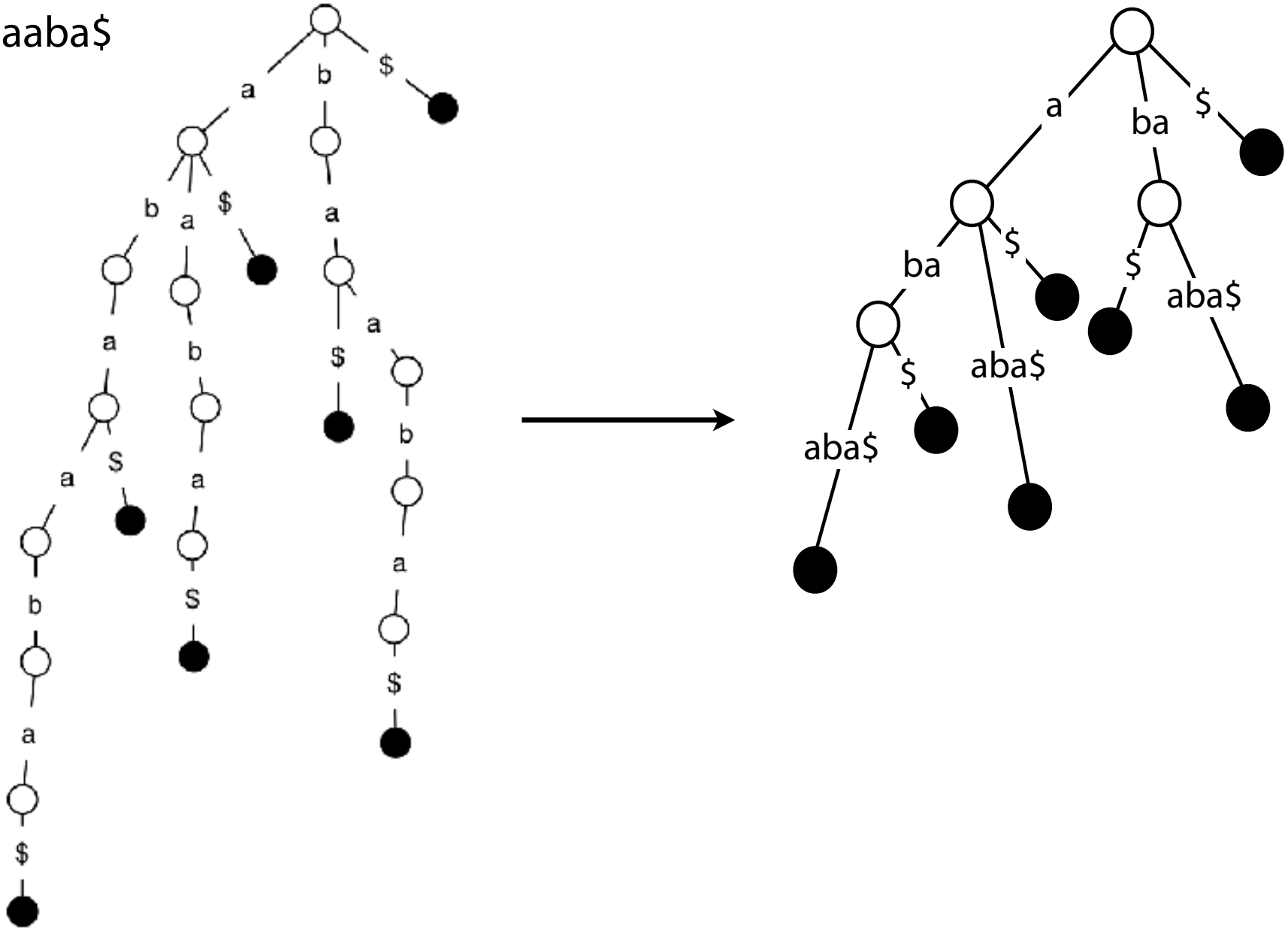
Idea 1: Coalesce non-branching paths into a *single edge* with a *string* label



Reduces # nodes, edges,
guarantees non-leaf nodes have >1 child

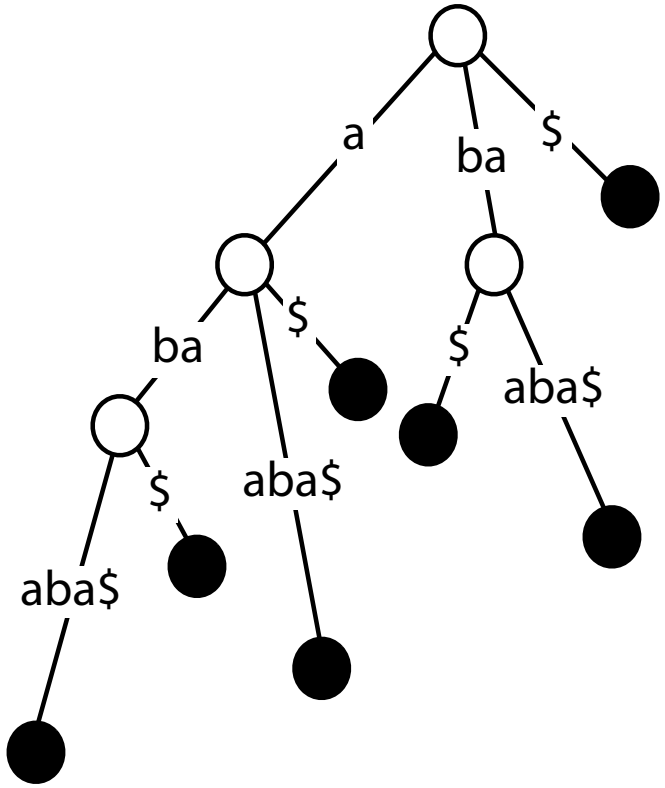
Suffix trie: making it smaller

T = abaaba$



Suffix tree

T = abaaba\$ $\quad |T| = m$



leaves? ***m***

non-leaf nodes (bound)? ***≤ m - 1***

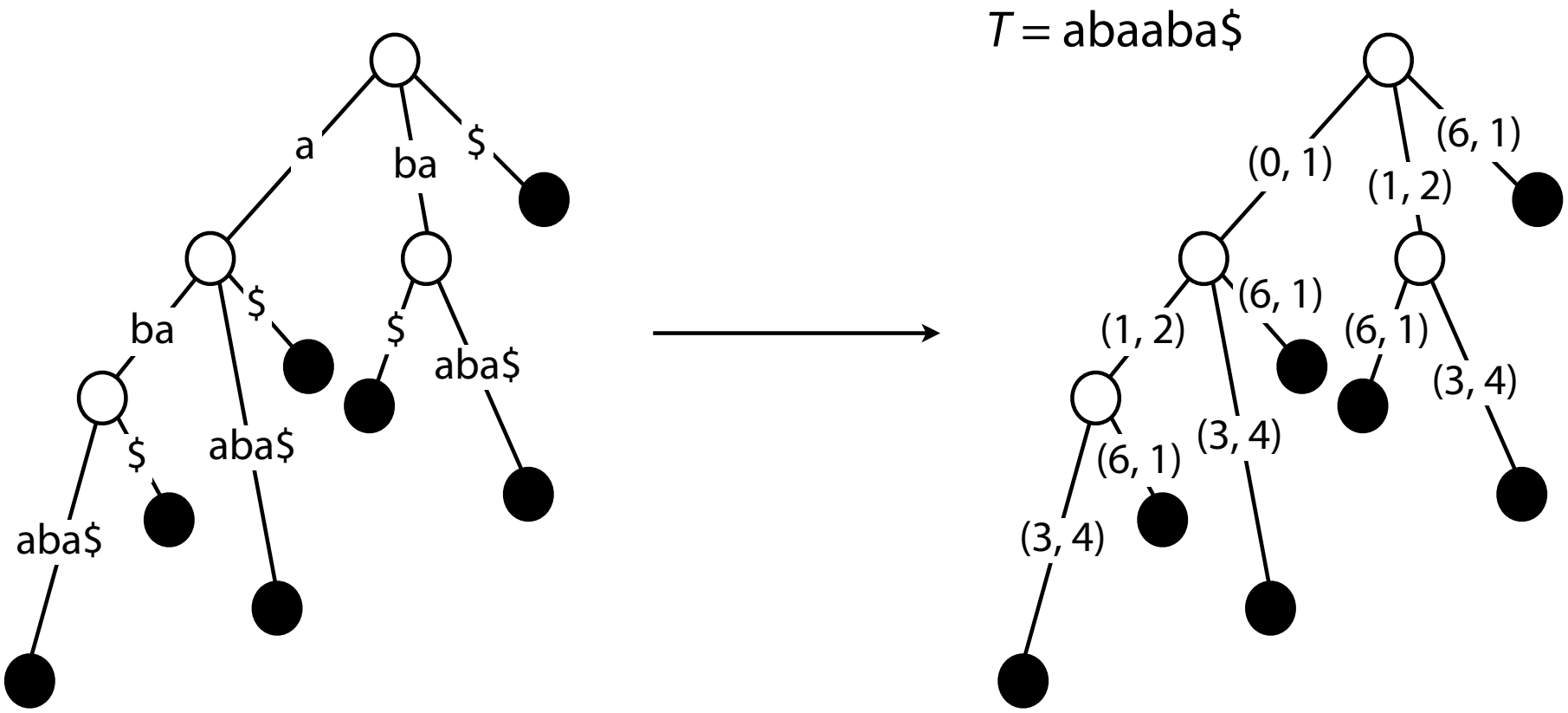
≤ 2m -1 nodes total — $O(m)$

Is *total size* $O(m)$ now?

No: total length of edge labels grows with m^2

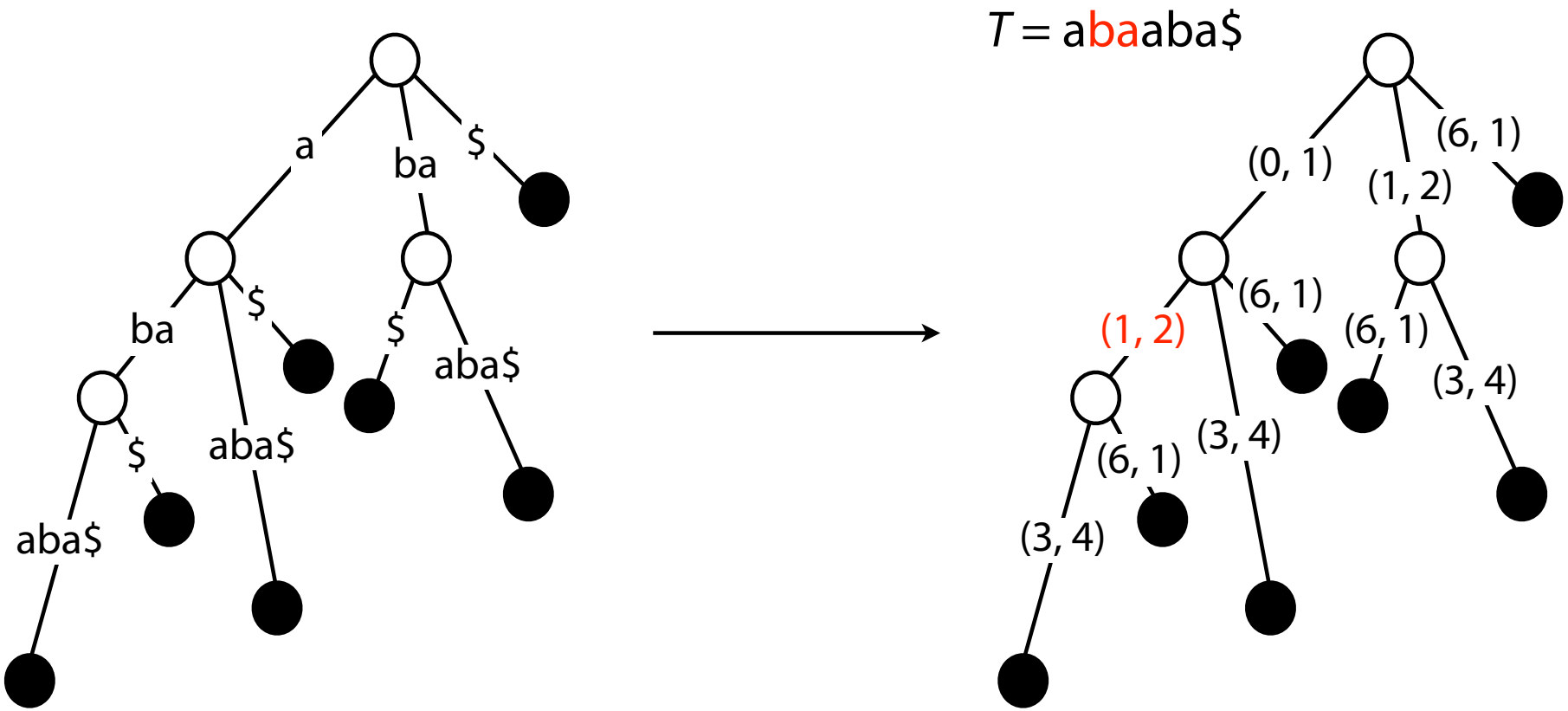
Suffix tree

Idea 2: Store *T* itself in addition to the tree. Convert tree's edge labels to (offset, length) pairs with respect to *T*.



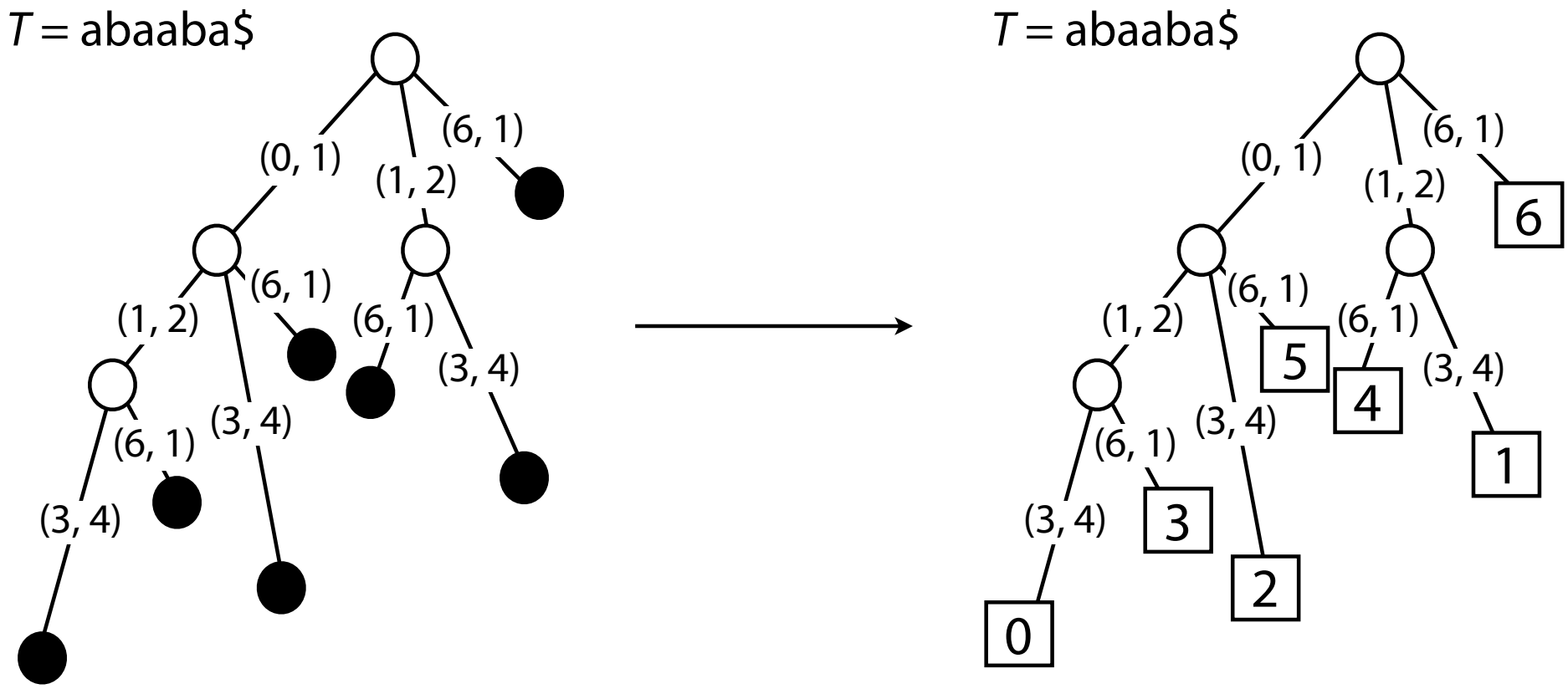
Suffix tree

Idea 2: Store *T* itself in addition to the tree. Convert tree's edge labels to (offset, length) pairs with respect to *T*.

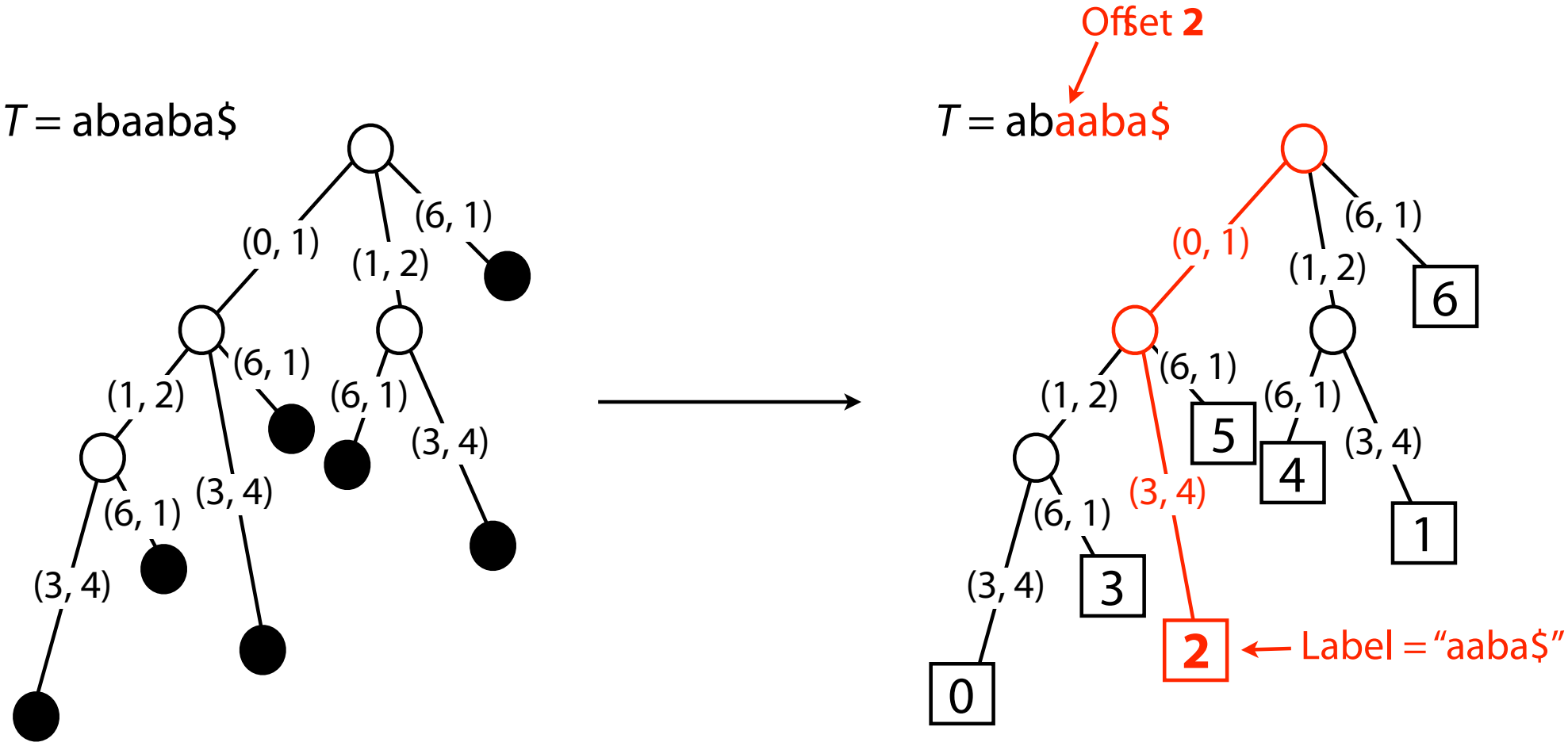


Space is now $O(m)$ Suffix trie was $O(m^2)$!

Suffix tree: leaves hold offsets



Suffix tree: leaves hold offsets



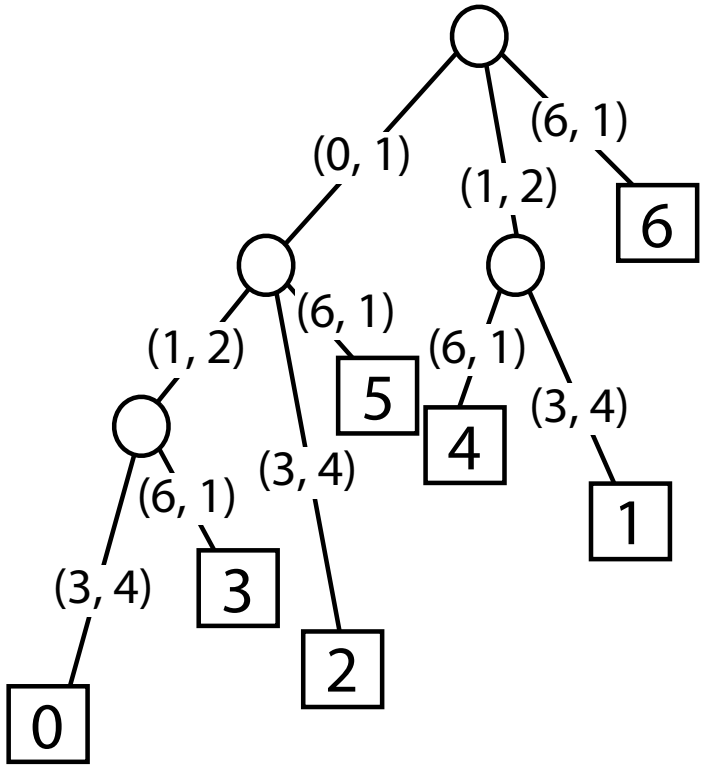
Suffix tree: building

Method 1: build suffix trie, coalesce non-branching paths, relabel edges

$O(m^2)$ time, $O(m^2)$ space

Method 2: build single-edge tree representing longest suffix, augment to include the 2nd-longest, augment to include 3rd-longest, etc (Gusfeld 5.4)

$O(m^2)$ time, $O(m)$ space



Suffix tree: building

Canonical method: Ukkonen's algorithm

> Ukkonen, Esko. "On-line construction of suffix trees."
> *Algorithmica* 14.3 (1995): 249-260.

$O(m)$ time and space!

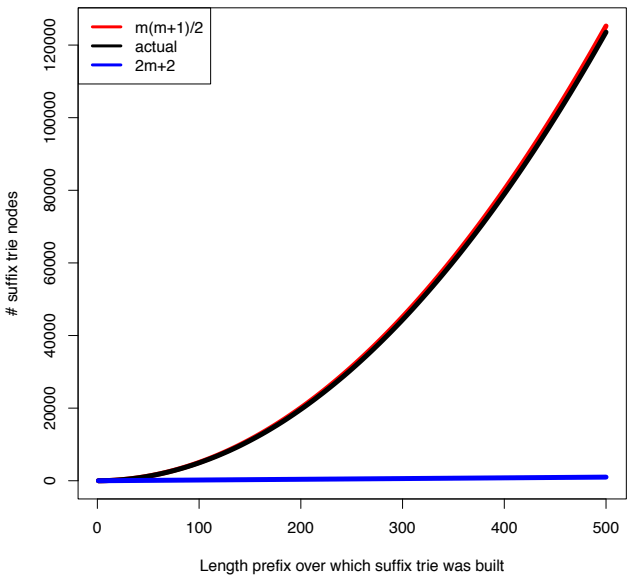
Won't cover it in class; see Gusfield Ch. 6 for details

Suffix tree: actual growth

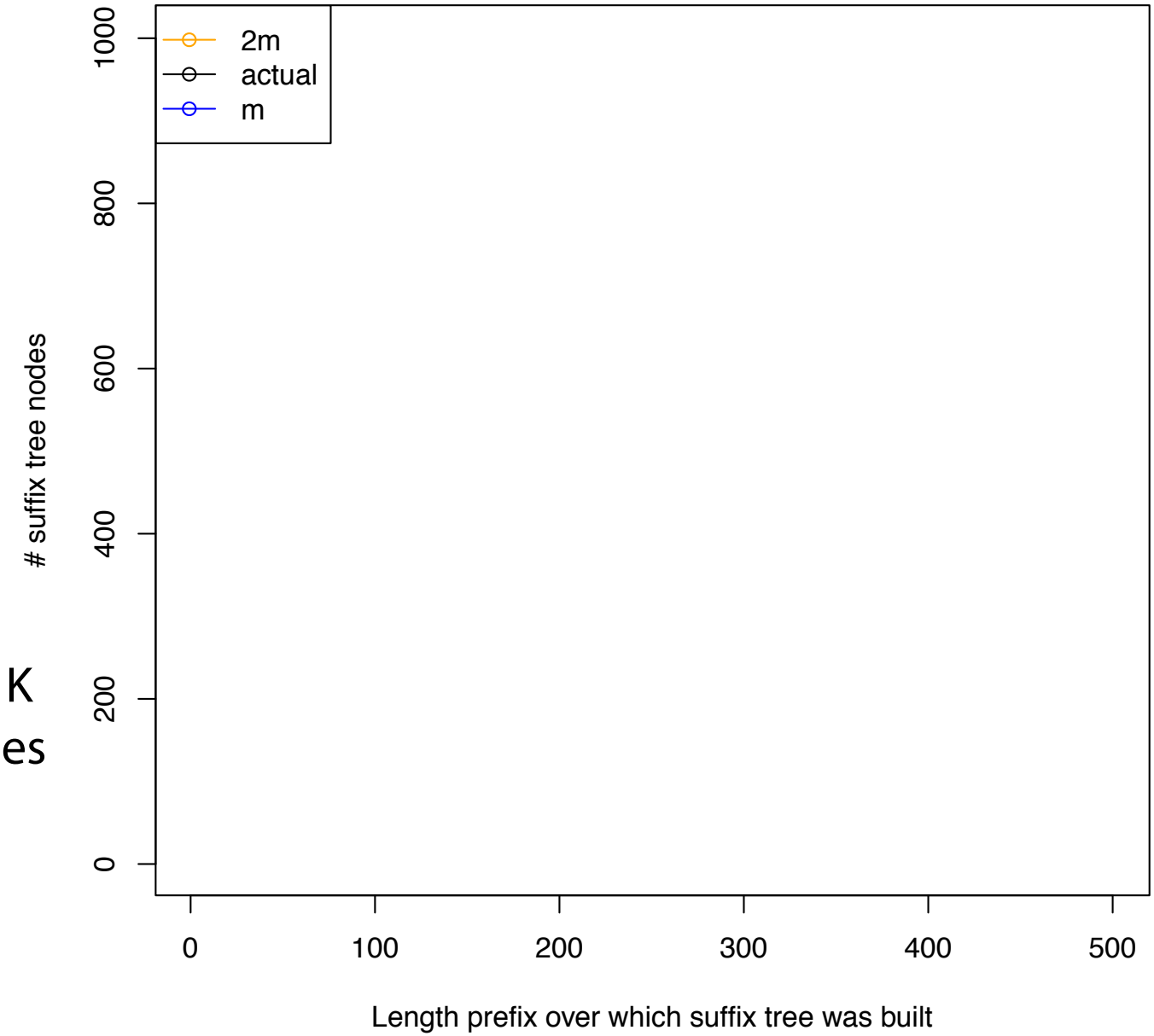
Built suffix trees for the first 500 prefixes of the lambda phage virus genome

Black curve shows # nodes increasing with prefix length

Remember suffix trie plot:



123 K nodes

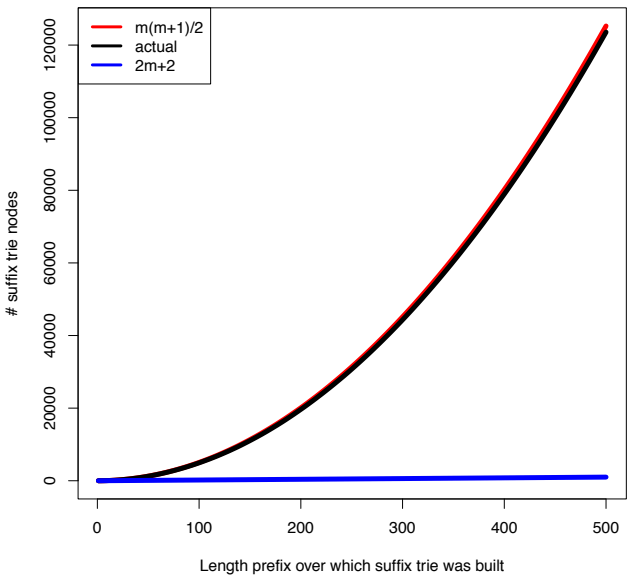


Suffix tree: actual growth

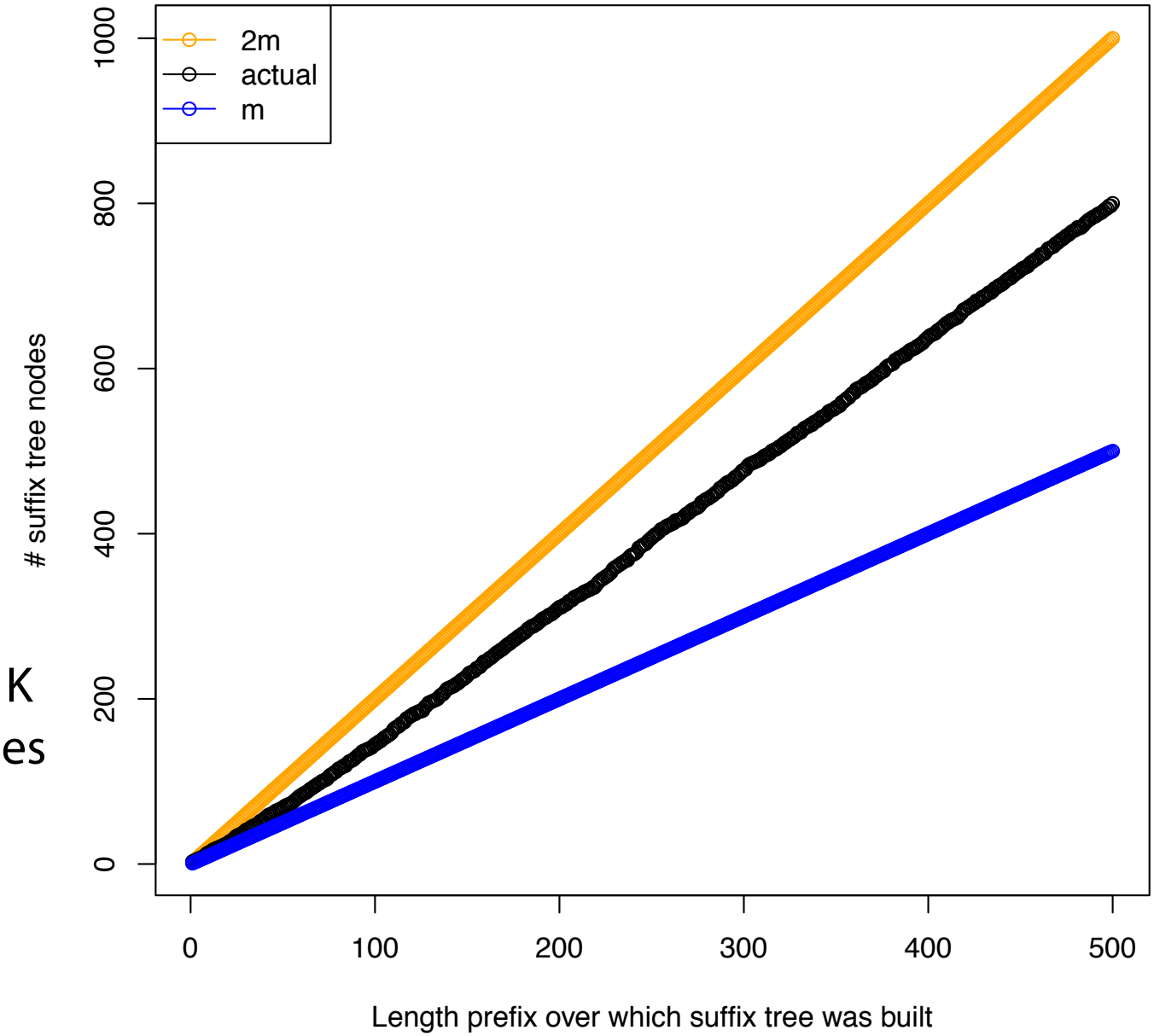
Built suffix trees for the first 500 prefixes of the lambda phage virus genome

Black curve shows # nodes increasing with prefix length

Remember suffix trie plot:



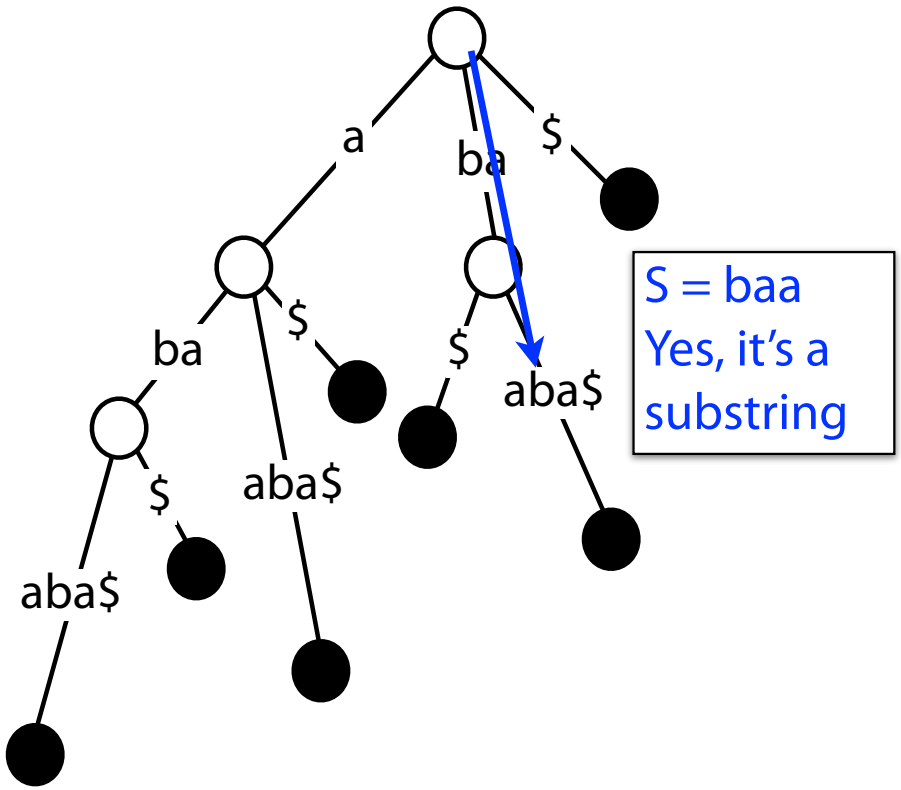
123 K nodes



Suffix tree

How do we check whether a string *S* is a substring of *T*?

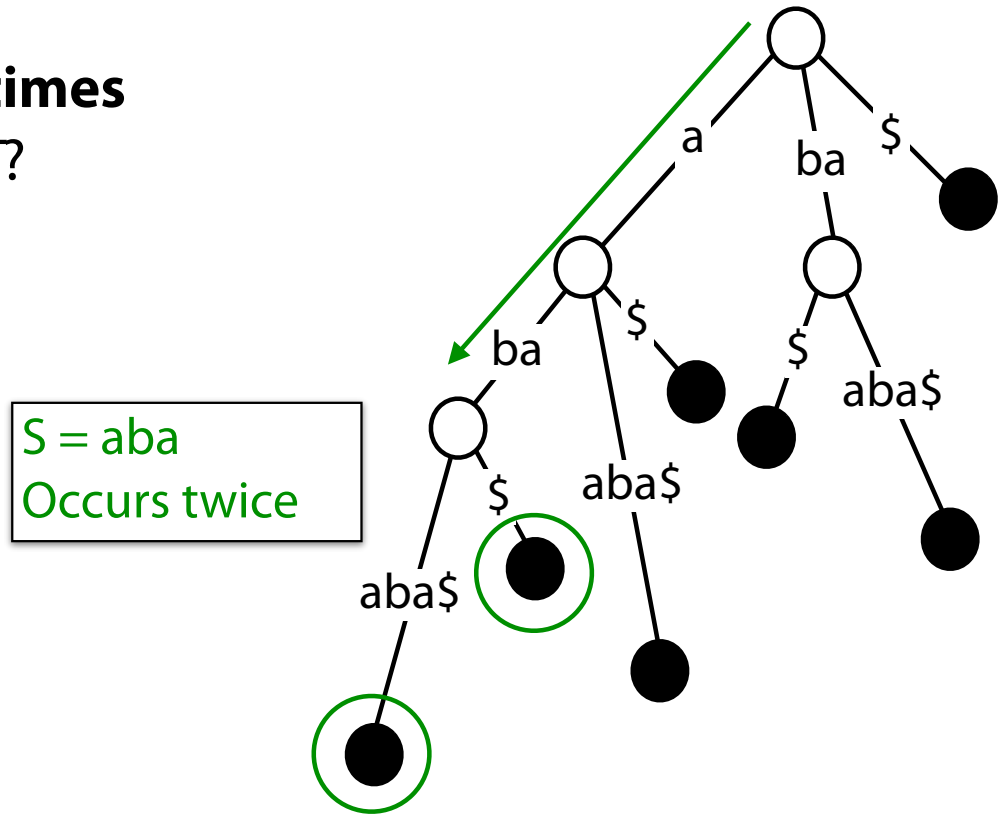
Same procedure as for suffix trie, but we have to deal with coalesced edges



Suffix tree

How do we count the **number of times** a string *S* occurs as a substring of *T*?

Same procedure as for suffix trie



Suffix tree

We can also **count or find** all the matches of *P* to *T*. Let *k* = # matches.

E.g., *P* = ab, *T* = abaaba$

O(*n*) Step 1: walk down ab path

If we "fall off" there are no matches

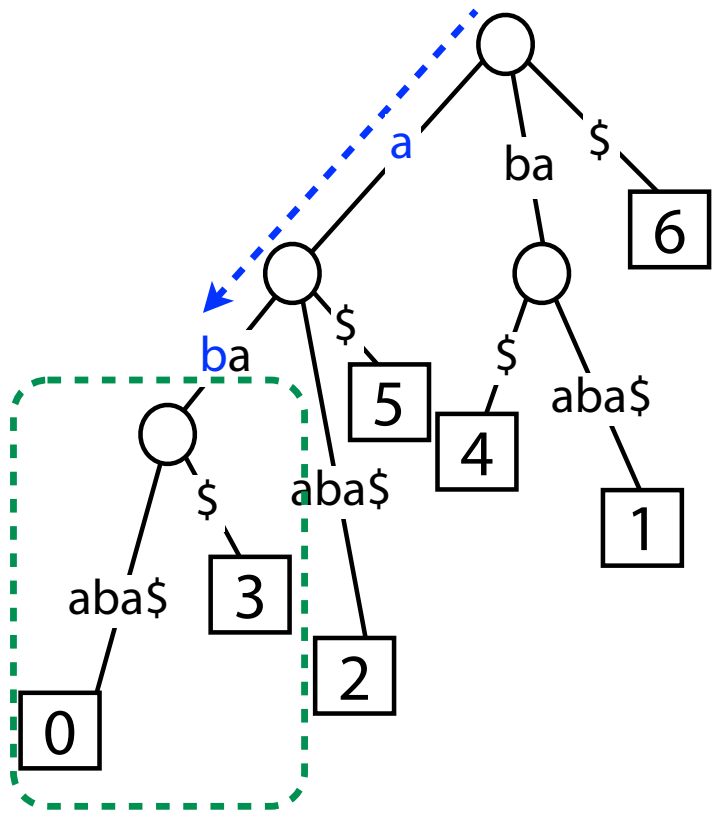
O(*k*) Step 2: visit all leaf nodes below

Report each leaf offset as match offset

leaves in subtree is is k,

non-leaves is ≤ k-1

O(*n* + *k*) time overall



abaaba

ab ab

Suffix trees in the real world: the constant factor

$O(m)$ is desirable, but "constant factor" is significant, sometimes making the suffix tree inconvenient

Constant factor varies depending on implementation:

MUMmer constant factor = 3.94 GB / 250 million nt ≈ **15.76 bytes per nt**

Kurtz, Stefan. "Reducing the space requirement of suffix trees." *Software Practice and Experience* 29.13 (1999): 1149-1171.

Suffix tree of human genome will be >45GB, perhaps much larger depending on exact data structures underlying suffix tree nodes/edges

Suffix arrays

Ben Langmead

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

Department of Computer Science

CC BY

Please sign guestbook (www.langmead-lab.org/teaching-materials) to tell me briefly how you are using the slides. For original Keynote fles, email me (ben.langmead@gmail.com).

Suffix array

```
T = abaaba$
    0123456
```

As with suffix tree, *T* is part of index

SA(*T*) =

6	$
5	a $
2	a a b a $
3	a b a $
0	a b a a b a $
4	b a $
1	b a a b a $

m integers

Suffix array of *T* is an array of integers in [0, *m*) specifying lexicographic (alphabetical) order of *T*'s suffixes

Suffix array: querying

Is *P* a substring of *T*?

T = abaaba$

1. For *P* to be a substring, it must be a prefix of ≥1 of *T*'s suffixes
2. Suffixes sharing a prefix are consecutive in the suffix array

Use binary search

6	$
5	a $
2	a a b a $
3	a b a $
0	a b a a b a $
4	b a $
1	b a a b a $

Suffix array: querying

Is P a substring of T?

Do binary search, check whether P is a prefix of the suffix there

Query time is $O(\ ?\ )$...

...$O(\log_2 m)$ bisections, $O(n)$ comparisons per bisection, so $O(n \log m)$

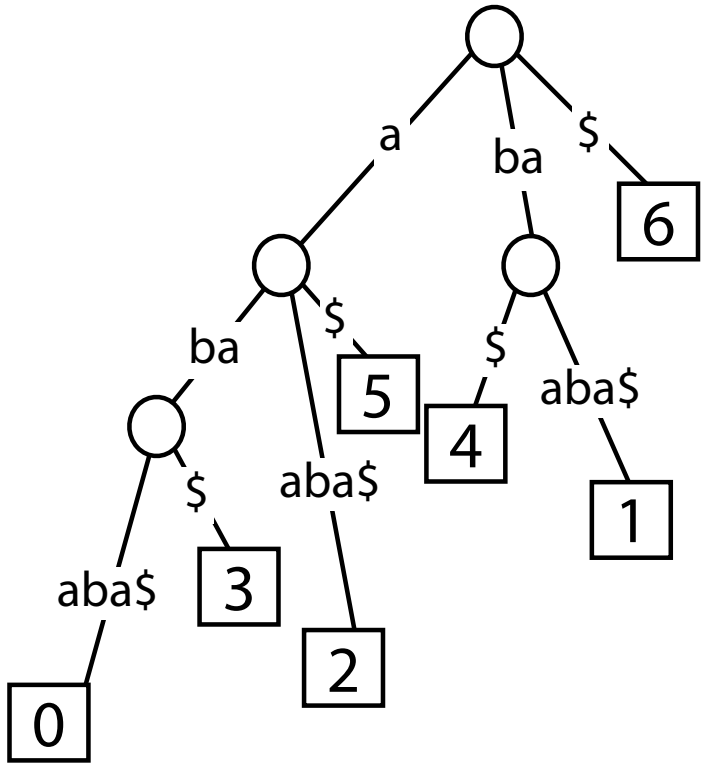
T = abaaba$

6	$
5	a $
2	a a b a $
3	a b a $
0	a b a a b a $
4	b a $
1	b a a b a $

Suffix array: querying

Contrast suffix array query time: O(n log m) with suffix tree: O(n)

6	$
5	a $
2	a a b a $
3	a b a $
0	a b a a b a $
4	b a $
1	b a a b a $

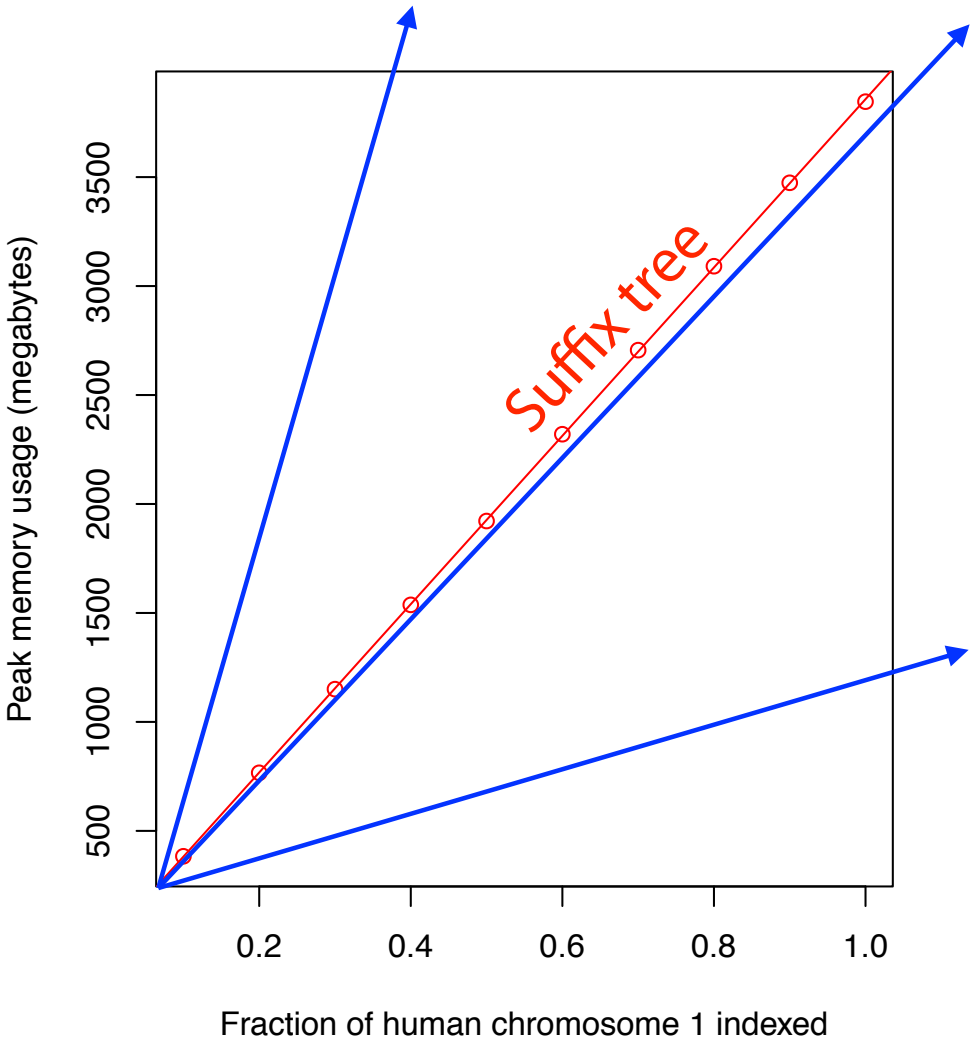


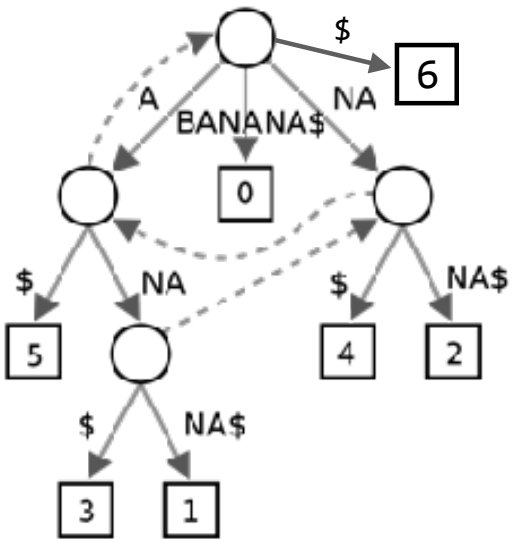
Time can be improved to O(n + log m), but we won't discuss here (See Gusfeld 7.17.4). For this class, we'll consider it O(n log m).

Suffix array

O(*m*) space, like suffix tree

Is "constant factor" worse, better, same?

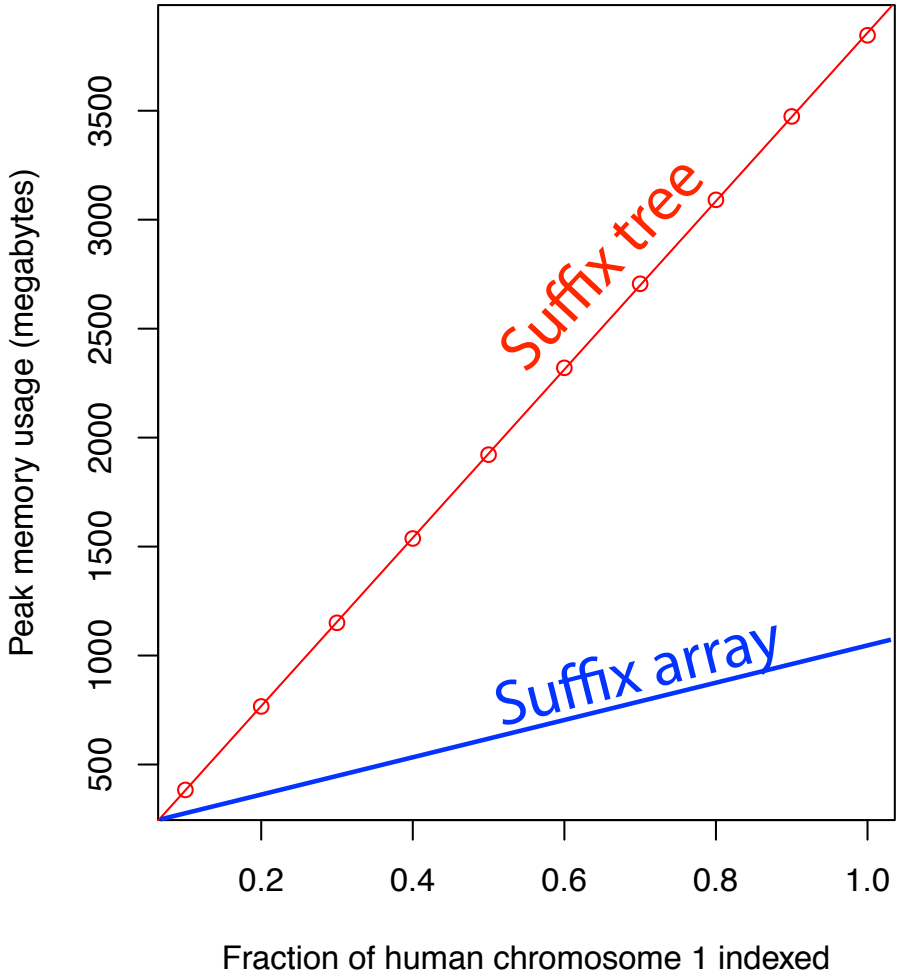


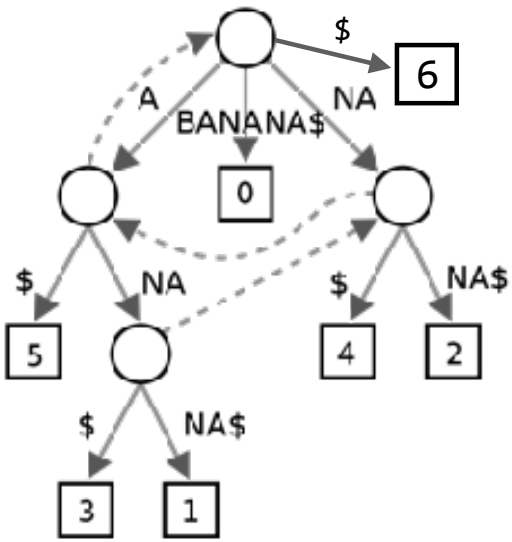


6	$
5	A$
3	ANA$
1	ANANA$
0	BANANA$
4	NA$
2	NANA$

Suffix array

32-bit integers sufficient for human genome, so fts in
~4 bytes/base × 3 billion bases ≈ 12 GB. Suffix tree is >45 GB.





6	$
5	A$
3	ANA$
1	ANANA$
0	BANANA$
4	NA$
2	NANA$

Suffix array: sorting suffixes

Use your favorite sort, e.g., quicksort

0	a b a a b a $
1	b a a b a $
2	a a b a $
3	a b a $
4	b a $
5	a $
6	$

```
def quicksort(q):
    lt, gt = [], []
    if len(q) <= 1:
        return q
    for x in q[1:]:
        if x < q[0]:
            lt.append(x)
        else:
            gt.append(x)
    return quicksort(lt) + q[0:1] + quicksort(gt)
```

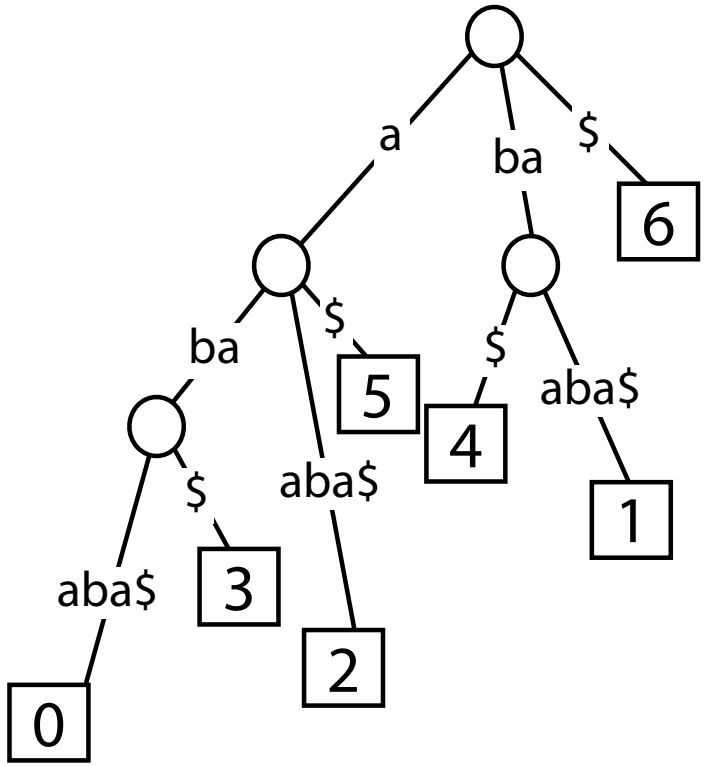
Expected time: O($m^2 \log m$)

Not $O(m \log m)$ because a suffix comparison is $O(m)$ time

Suffix array: building

How to build a suffix array?

6	$
5	a $
2	a a b a $
3	a b a $
0	a b a a b a $
4	b a $
1	b a a b a $



(a) Build suffix tree, (b) traverse in alphabetical order,
(c) upon reaching leaf, append suffix to array

Suffix array: sorting suffixes

Another idea: Use a sort algorithm that's aware that the items being sorted are all suffixes of the same string

- Original suffix array paper suggested an $O(m \log m)$ algorithm

  Manber U, Myers G. "Suffix arrays: a new method for on-line string searches." SIAM Journal on Computing 22.5 (1993): 935-948.

- Other popular $O(m \log m)$ algorithms have been suggested

  Larsson NJ, Sadakane K. Faster suffix sorting. Technical Report LU-CS-TR: 99-214, LUNDFD6/(NFCS-3140)/1-43/(1999), Department of Computer Science, Lund University, Sweden, 1999.

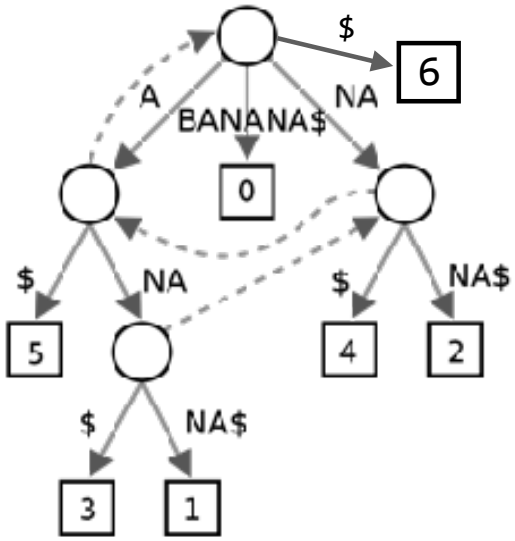
- More recently $O(m)$ algorithms have been demonstrated!

  Kärkkäinen J, Sanders P. "Simple linear work suffix array construction." Automata, Languages and Programming (2003): 187-187.

  Ko P, Aluru S. "Space efficient linear time construction of suffix arrays." *Combinatorial Pattern Matching*. Springer Berlin Heidelberg, 2003.

Suffix array: summary

Just *m* integers, with O(*n* log *m*) query time



Suffix Tree

6	$
5	A$
3	ANA$
1	ANANA$
0	BANANA$
4	NA$
2	NANA$

Suffix Array

Constant factor greatly reduced compared to suffix tree:
human genome index fts in ~12 GB instead of > 45 GB